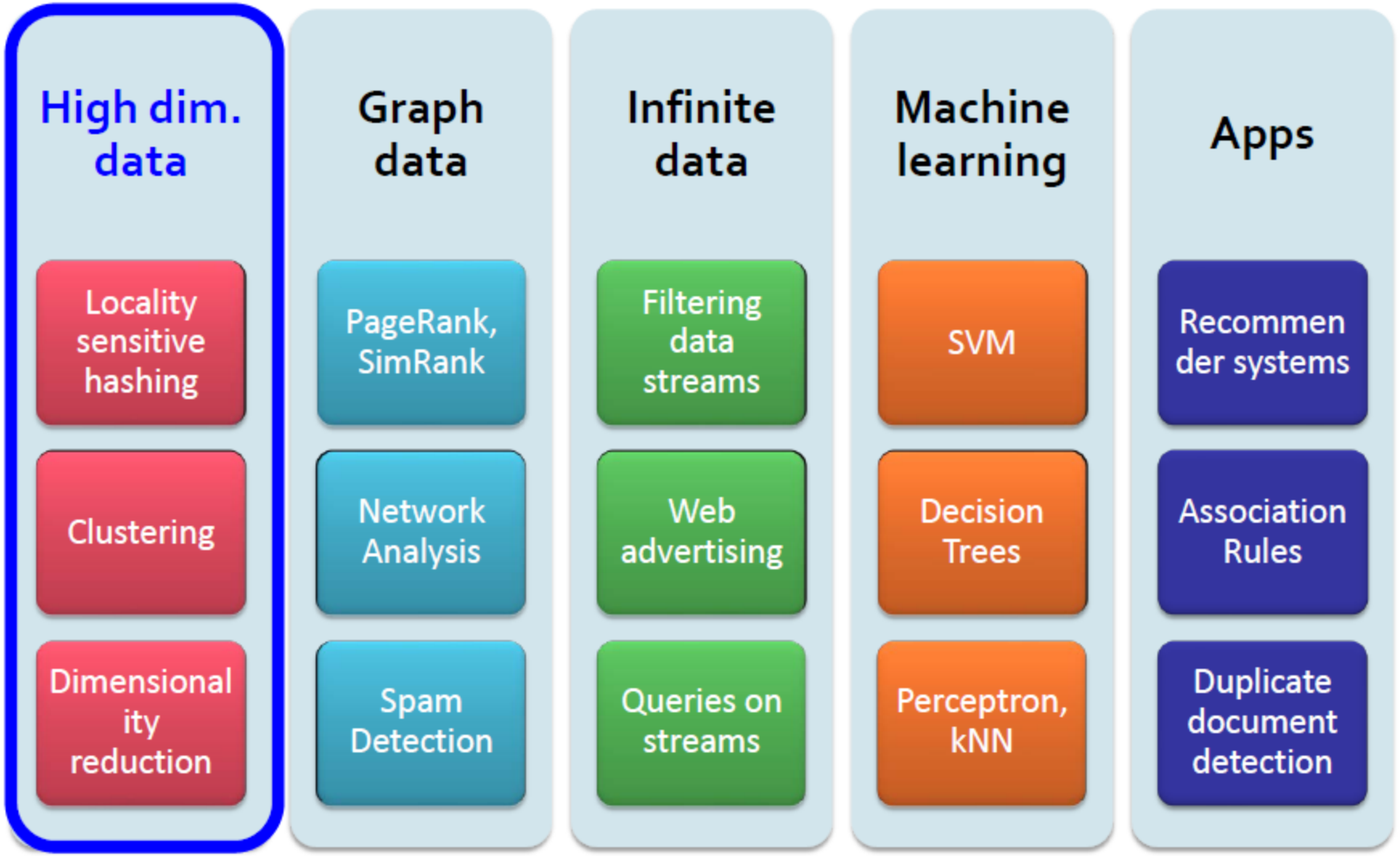
Big data analytic

Le Thanh Van

HCMC University of Technology

Finding similar items

High dimension data



Finding similar item

Google nhatho.jpg × ho chi minh city

Web Images News More Search tools

About 1,050 results (0.80 seconds)

Image size:
800 × 533

Find other sizes of this image:
All sizes - Medium

Best guess for this image: ***ho chi minh city***

Ho Chi Minh City - Wikipedia, the free encyclopedia
en.wikipedia.org/wiki/Ho_Chi_Minh_City

Ho Chi Minh City

City in Vietnam

Ho Chi Minh City, is the largest city in Vietnam. Under the name Saigon, it was the capital of the French colony of Cochinchina and later of the independent republic of South Vietnam from 1955–75. Wikipedia

Population: 7.396 million (2010) Vietnam General Statistics Office

Area: 2,095 km²

Founded: 1698

Weather: 29°C, Wind S at 23 km/h, 62% Humidity

Colleges and Universities: Saigon University, more

Points of interest


Independ…


War

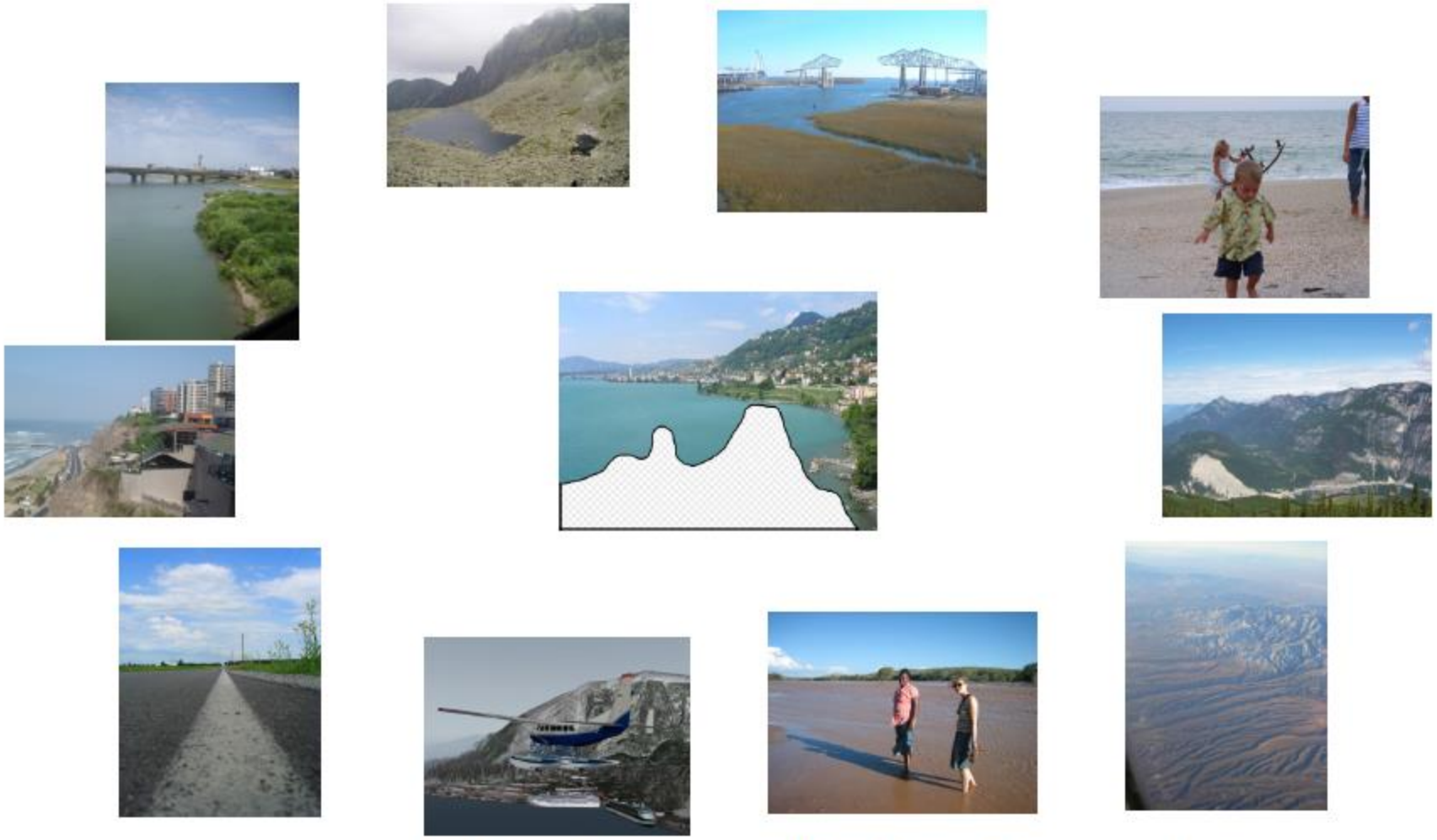

Cu Chi


Saigon


Pham Ngũ

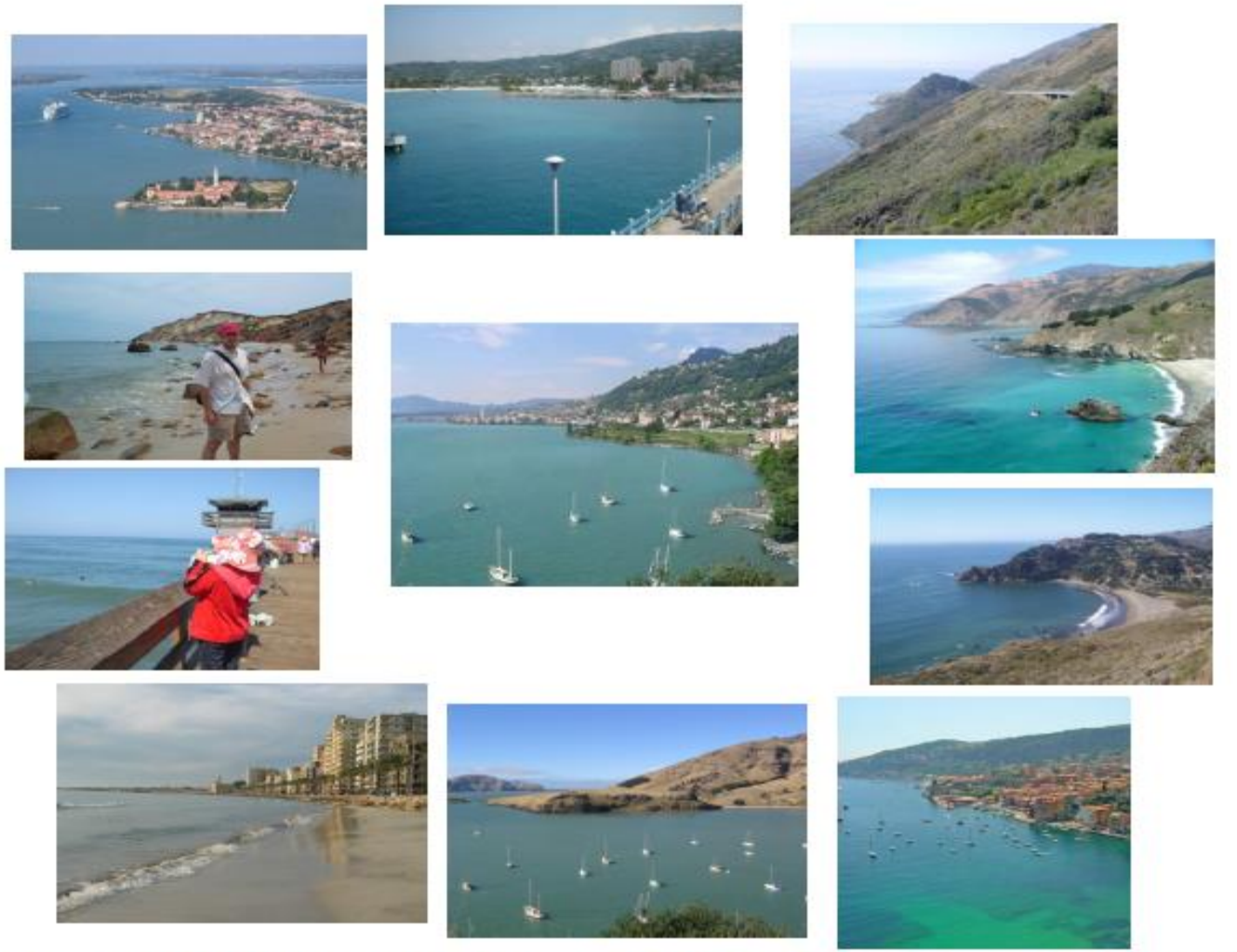
Finding similar image

Scene completion problem



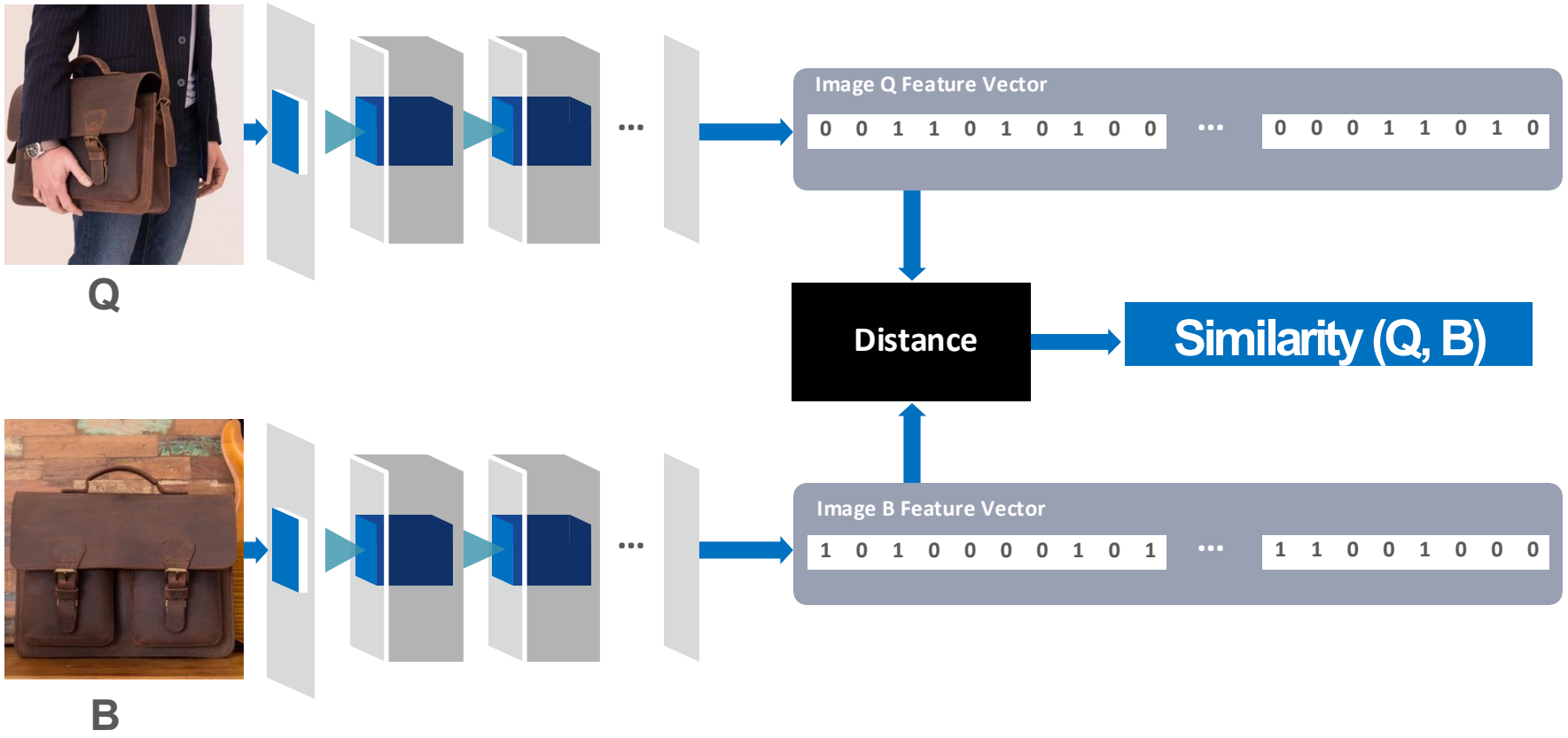
10 nearest neighbors from a collection of 20,000 images

Scene completion problem



10 nearest neighbors from a collection of 2 million images

How does it work?



- Collect billions of images
- Determine feature vector for each image (4k dim)
- **Given a query Q, find nearest neighbors FAST**

Finding similar item???

- Many problems can be expressed as finding “similar” sets:
  - Find near-neighbors in high-dimensional space
- Examples:
  - Pages with similar words
    - For duplicate detection, classification by topic
  - Customers who purchased similar products
    - Products with similar customer sets
  - Images with similar features
    - Users who visited similar websites

How to compute distance in large datasets?

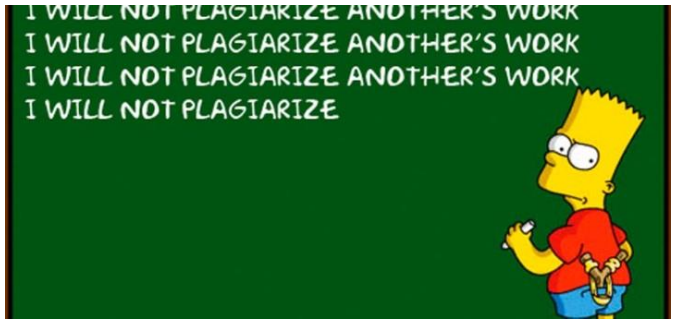
- Given: High dimensional data points $\boldsymbol{x_1}, \boldsymbol{x_2}, \ldots$
- For example: Image is a long vector of pixel colors

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 2 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

- $\rightarrow$[1 2 1 0 2 1 0 1 0]
- And some distance function $\boldsymbol{d(x_1, x_2)}$
  - Which quantifies the "distance" between $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$
- Goal: Find all pairs of data points $(\boldsymbol{x_i}, \boldsymbol{x_j})$ that are within some distance threshold $\boldsymbol{d(x_1, x_2) \leq s}$
- Note: Naïve solution would take $\boldsymbol{O(N^2)}$ where $\boldsymbol{N}$ is the number of data points
- MAGIC: This can be done in $\boldsymbol{O(N)}$!! How?

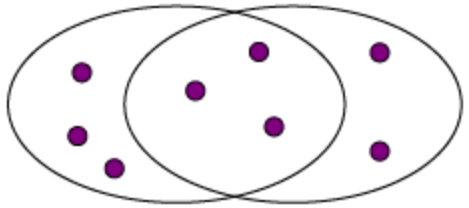
Find pairs of similar docs

- **Main idea: Candidates**
- **Pass 1:** Take document and hash them to buckets such that documents that are similar hash to the same bucket
- **Pass 2:** Only compare document that are candidate (i.e they hashed to a same bucket)
- **Benefits:** Instead of $O(N^2)$ comparisons, we need $O(N)$ comparisions to find similar documents



Distance measure

- **Goal: Find near-neighbors in datasets**
  - "near neighbors": points that are a "small distance" apart
- For each application, we first need to define what "**distance" means**
- **Jaccard distance/similarity**
  - The Jaccard **similarity** of two sets is the size of their intersection divided by the size of their union:
    - **$sim(C_1, C_2) = |C1 \cap C2| / |C_1 \cup C_2|$**
  - Jaccard distance:
    - **$d(C_1, C_2) = 1 - sim(C_1, C_2)$**



3 in intersection
8 in union
Jaccard similarity= 3/8
Jaccard distance = 5/8

Common Distance Measures

1. Euclidean Distance

$$d(p,q) = \sqrt{(\Sigma(p_i - q_i)^2)}$$

Use: k-NN, clustering in low dimensions

2. Manhattan Distance

$$d(p,q) = \Sigma|p_i - q_i|$$

Use: Grid-based movement, robust to outliers

3. Minkowski Distance

$$d(p,q) = (\Sigma|p_i - q_i|^p)^{1/p}$$

Use: p =1 (Manhattan), p =2 (Euclidean)

4. Cosine Similarity

$$\cos(\theta) = (A \cdot B)/(||A|| \cdot ||B||)$$

Use: Text analysis, high-dimensional data

5. Hamming Distance

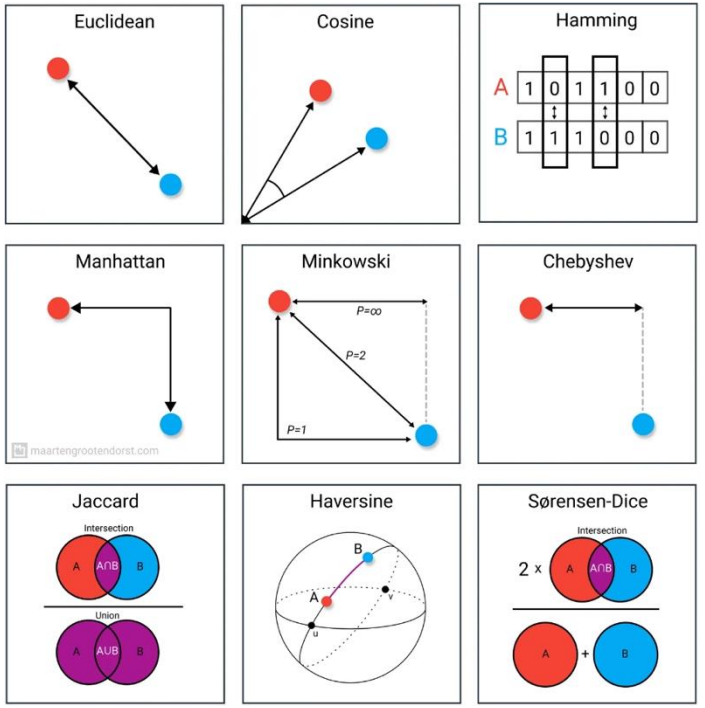
$$d(p, q) = \sum(p_i \neq q_i)$$

Use: Error detection, categorical variables

6. Chebyshev Distance

$$d(p,q) = \max_i|p_i - q_i|$$

Use: Chess king movement, warehouse logistics



Specialized Distance Measures

1. Jaccard Distance

Measures dissimilarity between sample sets

$$d(A,B) = 1 - J(A,B) = 1 - |A \cap B|/|A \cup B|$$

Use: Document similarity, recommendation systems

2. Haversine Distance

Measures distance between points on a sphere

$$d = 2r \cdot \arcsin(\sqrt{(\sin^2((\varphi_2-\varphi_1)/2) + \cos(\varphi_1)\cdot\cos(\varphi_2)\cdot\sin^2((\lambda_2-\lambda_1)/2)))}$$

Use: Geographic calculations, GPS applications

3. Sørensen-Dice Index

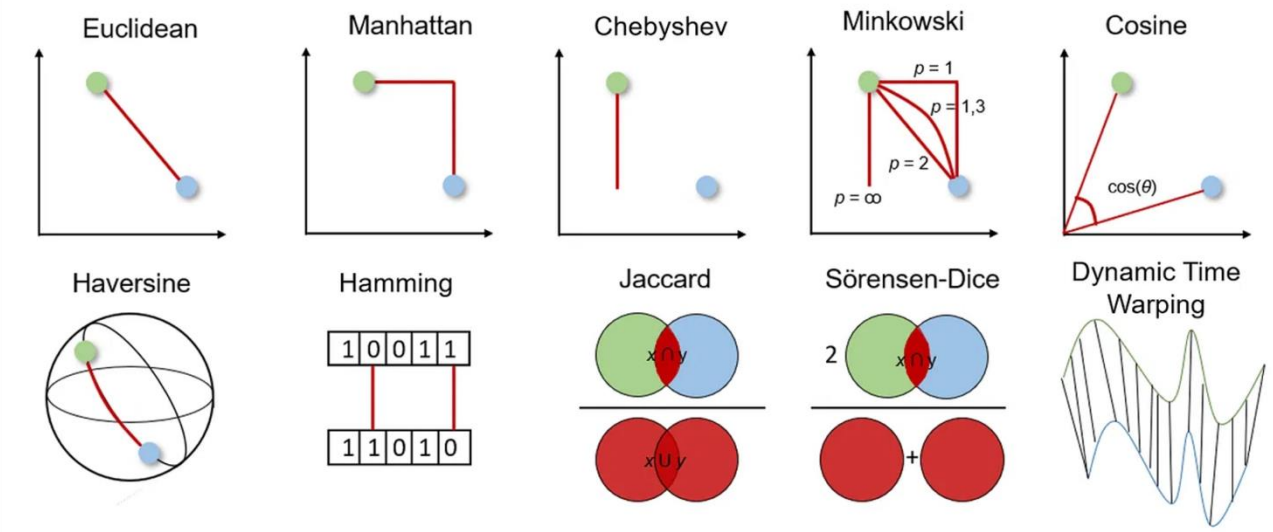
Measures overlap between two sets

$$DSC = 2|A \cap B|/(|A|+|B|)$$

Use: Image segmentation, ecological studies

Key Differences & Applications

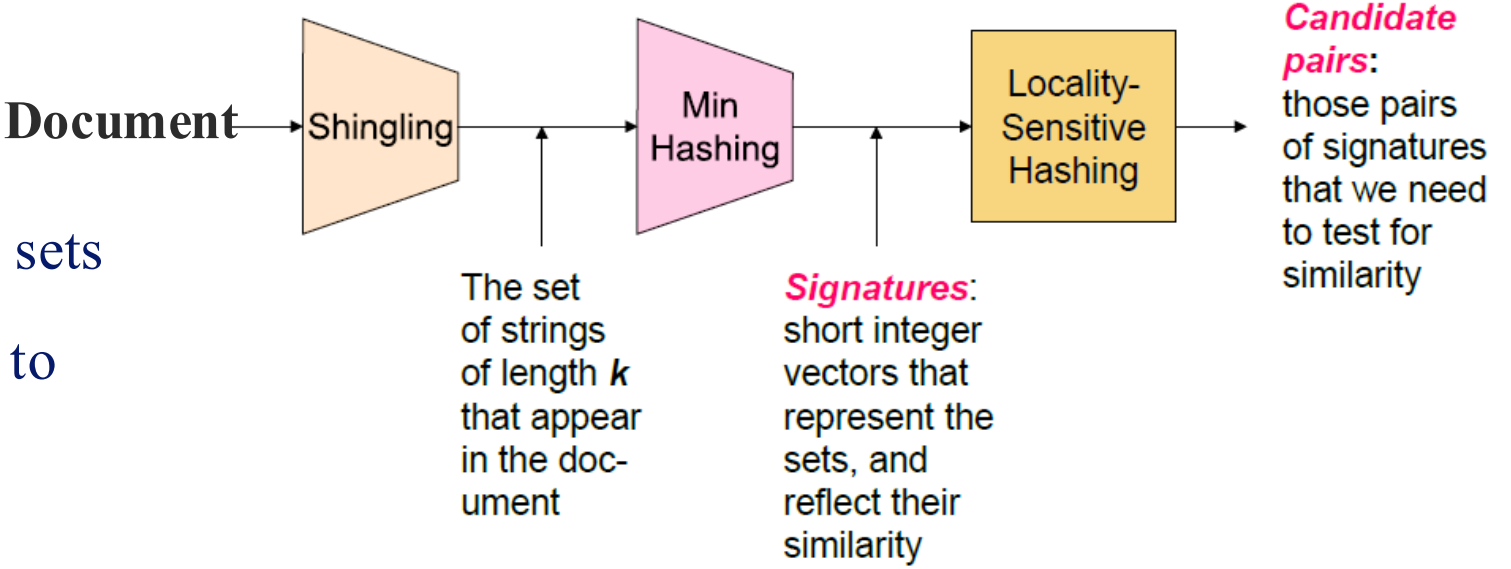
Jaccard: Focuses on set similarity, ignores element frequency
Haversine: Accounts for Earth's curvature, essential for navigation
Sørensen-Dice: Gives more weight to common elements than Jaccard



Finding similar documents in a large collection

- **Given a large number (in the millions or billions) of documents, find "near duplicate" pairs**
- **Applications:**
  - Mirror websites, or approximate mirrors
    - Don't want to show both in search results
  - Similar news articles at many news sites
    - Cluster articles by "same story"
- **Problems:**
  - Many small pieces of one document can appear out of order in another
  - Too many documents to compare all pairs
  - Documents are so large or so many that they cannot fit in main memory

3 essential steps for similar documents



1. ***Shingling:*** Convert documents to sets
2. ***Min-Hashing:*** Convert large sets to short signatures, while preserving similarity
3. ***Locality-Sensitive Hashing:*** Focus on pairs of signatures likely to be from similar documents

- **Candidate pairs!**

Shingling

- Convert documents to sets
  - Identify lexically similar documents: many common elements in documents sets
- **Simple approaches:**
  - Document = set of words appearing in document
  - Document = set of "important" words
  - Don't work well for this application. Why?
- **Need to account for ordering of words!**
  - A different way: **Shingles!**

Define: Shingles

- A *k*-shingle (or *k*-gram) for a document is a sequence of *k* tokens that appears in the doc
  - Tokens can be characters, words or something else, depending on the application
  - Assume tokens = characters for examples

- **Example: k=2**; document $\mathbf{D_1}$ = abcab
  Set of 2-shingles: $\mathbf{S(D_1)}$ = {ab, bc, ca}
  - **Option:** Shingles as a bag (multiset), count ab twice: $\mathbf{S'(D_1)}$ = {ab, bc, ca, ab}

k-Shingles

- Any substring of length k found within the document
  - abcdabd, k = 2 ⇒ 2-shingles sets {ab, bc, cd, da, bd}.
    - Bag of shingles {ab, bc, cd, da, ab, bd}
- **Handling Whitespaces:**

	With blanks	**Without blanks**
• "The plane was ready for touch down" ⇒	{the plane, …,touch dow, ouch down},	{…,touchdown}
• "The quarterback scored a touchdown" ⇒	{…,touchdown},	{…,touchdown}

Choosing the Shingle Size

- Idea:
  - **Documents that have lots of shingles in common have similar text, even if the text appears in different order**
- **k too small:** high similarity between documents
- Probability of any given k-shingle appearing in any given document is low
  - ***k = 5 is OK for short documents***
  - ***k = 10 is better for long documents***

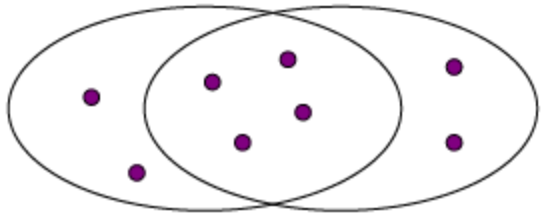
Hashing shingles

- To **compress long shingles, we can hash them to 4 bytes:**
  - hash strings of length k to some number of buckets in the range 0 to $2^{32} - 1$
- **Represent a document by the set of hash values of its *k-shingles***
  - *Idea: Two documents could (rarely) appear to have shingles in common, when in fact only the hash-values were shared*
- **Example: k=2; document D1= abcab**
  - Set of 2-shingles: S(D1) = {ab, bc, ca}
  - Hash the singles: h(D1) = {1, 5, 7}

Similarity metric for shingles

- **Document D_1 is a set of its k-shingles $C_1=S(D_1)$**
- Equivalently, each document is a 0/1 vector in the space of *k-shingles*
  - *Each unique shingle is a dimension*
  - Vectors are very sparse
- **A natural similarity measure is the Jaccard similarity:**
  - $sim(D1, D2) = |C1 \cap C2| / |C1 \cup C2|$

Element	Doc1	Doc2	Doc3	Doc4
a	1	0	1	0
b	0	1	0	0
c	0	1	0	1
d	1	1	1	1
e	0	0	1	0



From sets to boolean matrices

Encode sets using 0/1 (bit, Boolean) vectors

- **Rows** = elements (shingles)
- **Columns** = sets (documents)
  - 1 in row ***e*** and column ***s*** if and only if ***e*** is a member of ***s***
  - Column similarity is the Jaccard similarity of the corresponding sets (rows with value *1)*
  - **Typical matrix is sparse!**
- **Each document is a column:**
  - **Example: sim(C_1 ,C_2) = ?**
    - ✧ Size of intersection = 3; size of union = 6, Jaccard similarity (not distance) = 3/6
    - ✧ **d(C_1,C_2) = 1 – (Jaccard similarity) = 3/6**

Documents

Shingles

1	1	1	0
1	1	0	1
0	1	0	1
0	0	0	1
1	0	0	1
1	1	1	0
1	0	1	0

We don't really construct the matrix; just imagine it exists

Encoding sets as bit vectors

- Many similarity problems can be formalized as **finding subsets that have significant intersection**
  - One dimension per element in the universal set
- Interpret set intersection as bitwise **AND,** and set union as bitwise **OR**
- **Example: C_1 = 10111; C_2 = 10011**
  - Size of intersection = 3; size of union = 4,
  - Jaccard similarity (not distance) = 3/4
  - Distance: $d(C_1,C_2)$ = 1 – (Jaccard similarity) = 1/4

Challenges

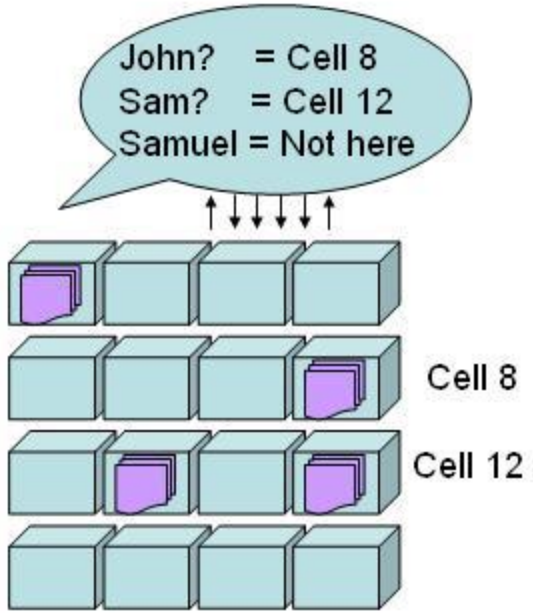
- Convert large sets into short signatures, while preserving similarity
- **Suppose we need to find near-duplicate documents among N=1 million documents**
- Naïvely, we would have to compute **pairwise Jaccard similarities for every pair of docs**
- **$N(N-1)/2 \approx 5*10^{11}$ comparisons**
- At 10^5 secs/day and 10^6 comparisons/sec, it would take **5 days**
- For N = 10 million, it takes more than a year… ☹

Finding similar columns

- **Naïve approach:**
  1. Signatures of columns: small summaries of columns
  2. Examine pairs of signatures to find similar columns
     - **Essential: Similarities of signatures and columns are related**
  3. Optional: Check that columns with similar signatures are really similar
- **Warnings: Comparing all pairs may take too much time: Job for LSH**
  - These methods can produce false negatives, and even false positives (if the optional check is not made)

Signatures (Hash columns)

- "hash" each column ***C*** to a small **signature** h(C), such that:
  1. h(C) is small enough that the signature fits in RAM
  2. sim(C1, C2) is the same as the "similarity" of signatures h(C1) and h(C2)
- Goal: **Find a hash function *h(·) such that:***
  - If ***sim(C1,C2)*** **is high, then with high prob.** ***h(C1) = h(C2)***
  - If ***sim(C1,C2)*** **is low, then with high prob.** ***h(C1) ≠ h(C2)***



- **Hash docs into buckets. Expect that "most" pairs of near duplicate docs hash into the same bucket!**

Min-hashing

- **Goal: Find a hash function *h(·) such that:***
  - *if sim(C1,C2) is high, then with high prob. h(C1) = h(C2)*
  - if *sim(C1,C2) is low, then with high prob. h(C1) ≠ h(C2)*
- **Clearly, the hash function depends on the similarity metric:**
  - Not all similarity metrics have a suitable hash function
- There is a suitable hash function for the Jaccard similarity: **It is called Min-Hashing**
- **Min-hashing property:**

$$\mathbf{Pr}[h_\pi(C1) = h_\pi(C2)] = sim(C1, C2)$$

where $\boldsymbol{\pi}$*, a random permutation*

Min-Hashing: Overview

- Permute the rows of the Boolean matrix using some permutation π
  - Thought experiment – not real
- Define min-hash function for this permutation π, $\mathbf{h_\pi(C)}$ = the number of the first (in the permuted order) row in which column *C* has value 1
  - Denoted this as: $\boldsymbol{h_\pi(C) = min_\pi \pi(C)}$
- Apply, to all columns, several randomly chosen permutations π to create a signature for each column
- **Result is a signature matrix:** Columns = sets, Rows = minhash values for each permutation π
- Use several (e.g., 100) independent hash functions (that is, permutations) to create a signature of a column

Similarity for signature

- We know: **Pr[*hπ(C1) = hπ(C2)] = sim(C1, C2)***
- Now generalize to multiple hash functions
- **The *similarity of two signatures is the fraction of the hash functions in which they agree***
- **Note:**

Because of the Min-Hash property, the similarity of columns is the same as the expected similarity of their signatures

The Min-Hash property

- **Choose a random permutation π**
- **Claim: Pr[$h_\pi(C_1) = h_\pi(C_2)$] = *sim*(C_1, C_2)**
- **Why?**
  - Let **X** be a doc (set of shingles), ***y*** ∈ ***X*** is a shingle
  - **Then: Pr[π(y) = min(π(X))] = 1/|X|**
    - ✧ It is equally likely that any ***y*** ∈ ***X*** is mapped to the ***min*** element
  - Let ***y*** be s.t. π(y) = min(π($C_1 \cup C_2$))
  - **Then either:** π(y) = min(π(C_1)) if y ∈ C_1 , **or**
    π(y) = min(π(C_2)) if y ∈ C_2
  - So the prob. that **both** are true is the prob. ***y*** ∈ $C_1 \cap C_2$
  - **Pr[min(π(C_1))=min(π(C_2))]=|$C_1 \cap C_2$|/|$C_1 \cup C_2$|= *sim*(C_1, C_2)**

0	0
0	0
1	**1**
0	0
0	1
1	0

One of the two cols had to have 1 at position ***y***

Four types of rows

- **Given cols C_1 and C_2, rows may be classified as:**

	C_1	C_2
A	1	1
B	1	0
C	0	1
D	0	0

  - **Define: a** = # rows of type A, etc.
- **Note: sim(C_1, C_2) = a/(a +b +c)**
- **Then: Pr[$h(C_1) = h(C_2)$] = $Sim(C_1, C_2)$**
  - Look down the cols C_1 and C_2 until we see a 1
  - If it's a type-*A* row, then $h(C_1) = h(C_2)$

    If a type-*B* or type-*C* row, then not

Similarity for Signatures

- We know: **Pr[$h_\pi(C_1) = h_\pi(C_2)$] = $sim(C_1, C_2)$**
- Now generalize to multiple hash functions

- **The *similarity of two signatures* is the fraction of the hash functions in which they agree**

- **Note:** Because of the Min-Hash property, the similarity of columns is the same as the expected similarity of their signatures

Min-Hash Signatures

- **Pick K=100 random permutations of the rows**
- Think of ***sig*(C)** as a column vector
- ***sig*(C)[i]** = according to the *i*-th permutation, the index of the first row that has a 1 in column *C*

$$sig(C)[i] = \min(\pi_i(C))$$

- **Note:** The sketch (signature) of document *C* is small **~100 bytes!**

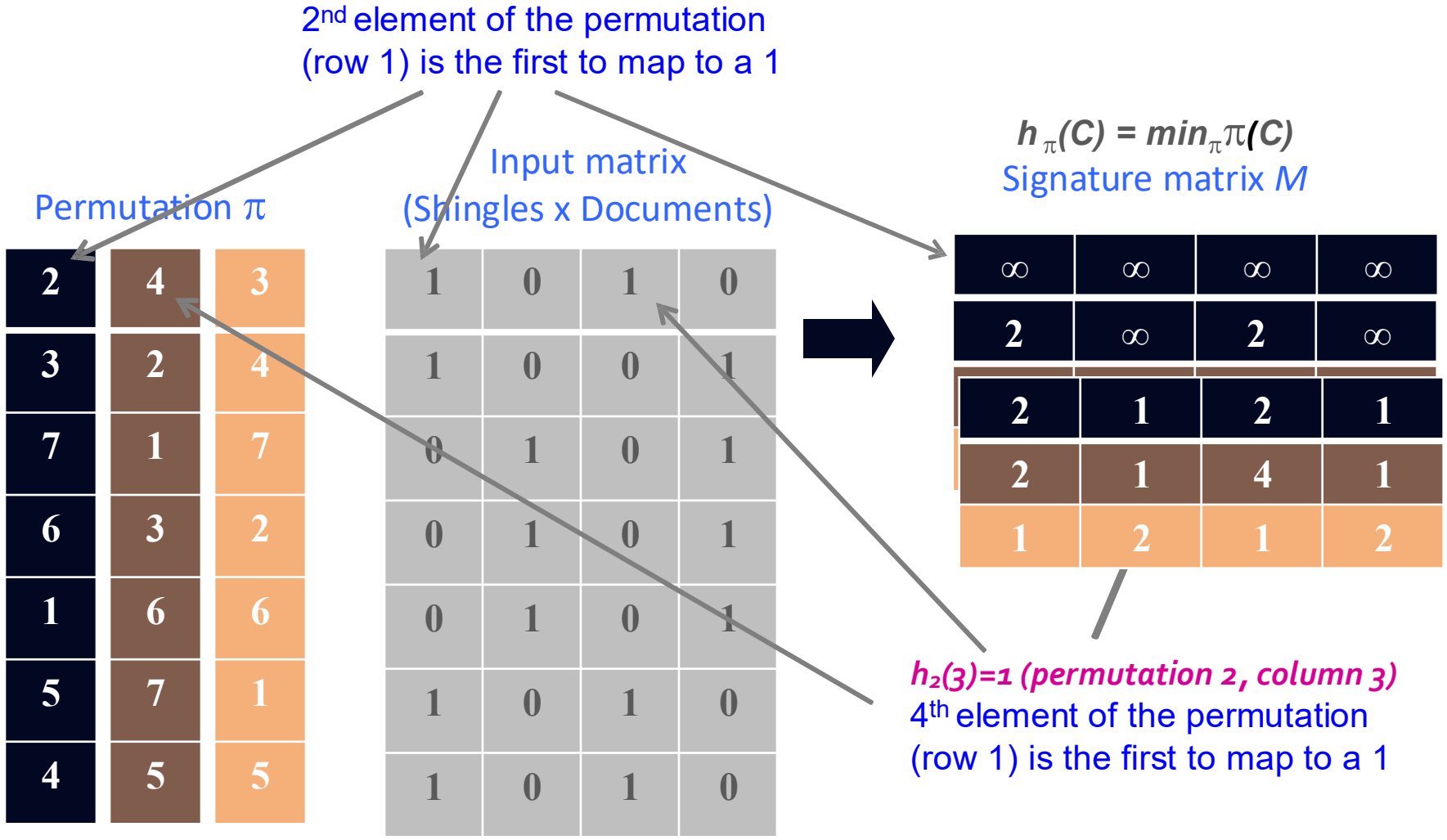
- **We achieved our goal! We "compressed" long bit vectors into short signatures**

Implementation Trick

- **Permuting rows even once is prohibitive**
- **Row hashing!**
  - Pick **K = 100** hash functions k_i
  - Ordering under k_i gives a random row permutation!
- **One-pass implementation**
  - For each column C and hash-func. k_i keep a "slot" for the min-hash value
  - Initialize all $sig(C)[i] = \infty$
  - **Scan rows looking for 1s**
    - Suppose row j has 1 in column C
    - Then for each k_i:
      - If $k_i(j) < sig(C)[i]$, then $sig(C)[i] \leftarrow k_i(j)$

How to pick a random hash function h(x)?
Universal hashing:
$h_{a,b}(x)=((a \cdot x+b) \bmod p) \bmod N$
where:
a,b … random integers
p … prime number (p > N)

Min-Hashing example



Minhashing example

- ***Min-Hashing***: Convert large sets into short signatures, while preserving similarity: $\Pr[h(C_1) = h(C_2)] = sim(D_1, D_2)$

Permutation π

2	4	3
3	2	4
7	1	7
6	3	2
1	6	6
5	7	1
4	5	5

Input matrix (Shingles x Documents)

1	0	1	0
1	0	0	1
0	1	0	1
0	1	0	1
0	1	0	1
1	0	1	0
1	0	1	0

Signature matrix M

2	1	2	1
2	1	4	1
1	2	1	2

Similarities of columns and signatures (approx.) match!

	1-3	2-4	1-2	3-4
Col/Col	0.75	0.75	0	0
Sig/Sig	0.67	1.00	0	0

Exercise

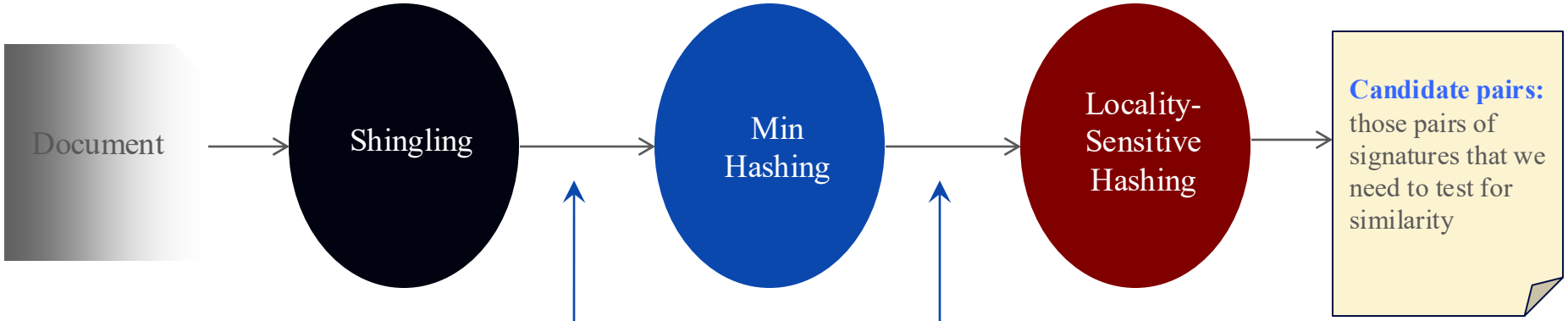
- Add to the signatures of the columns the values of the following hash functions:

  (a) $h_3(x) = (2x + 1) \bmod 6$.

  (b) $h_4(x) = (3x + 2) \bmod 6$.

Element	S_1	S_2	S_3	S_4
0	0	1	0	1
1	0	1	0	0
2	1	0	0	1
3	0	0	1	0
4	0	0	1	1
5	1	0	0	0

Step 3: Locality Sensitive Hashing



The set of strings of length k that appear in the document

Signature: Short integer vectors that represent the sets, and reflect their similarity

Focus on pairs of signatures likely to be from similar documents

Locality sensitive hashing

- Focus on pairs of signatures likely to be from similar documents
- **Goal:** Find documents with Jaccard similarity at least *s (for some similarity threshold, e.g., s=0.8)*
- **LSH – General idea:** Use a function ***f(x,y)*** that tells whether ***x*** and ***y*** is a candidate pair: a pair of elements whose similarity must be evaluated
- **For Min-Hash matrices: Hash columns of signature matrix *M to many buckets***
- Each pair of documents that hashes into the same bucket is a **candidate pair**

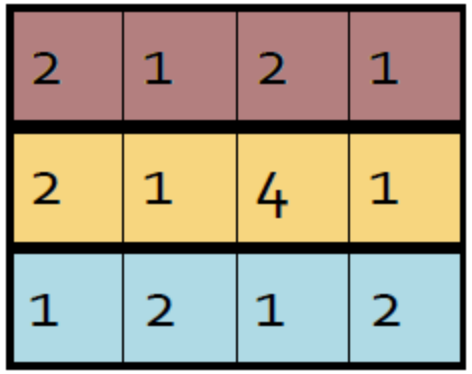
2	1	4	1
1	2	1	2
2	1	2	1

Candidates from Min-Hash

- **Pick a similarity threshold *s*** *(0 < s < 1)*
- Columns x and y of M are a candidate pair if their signatures agree on at least fraction ***s*** of their rows:

  M (i, x) = M (i, y) for at least frac. s values of i

- ***Expect documents x and y to have the same (Jaccard) similarity as their signatures***



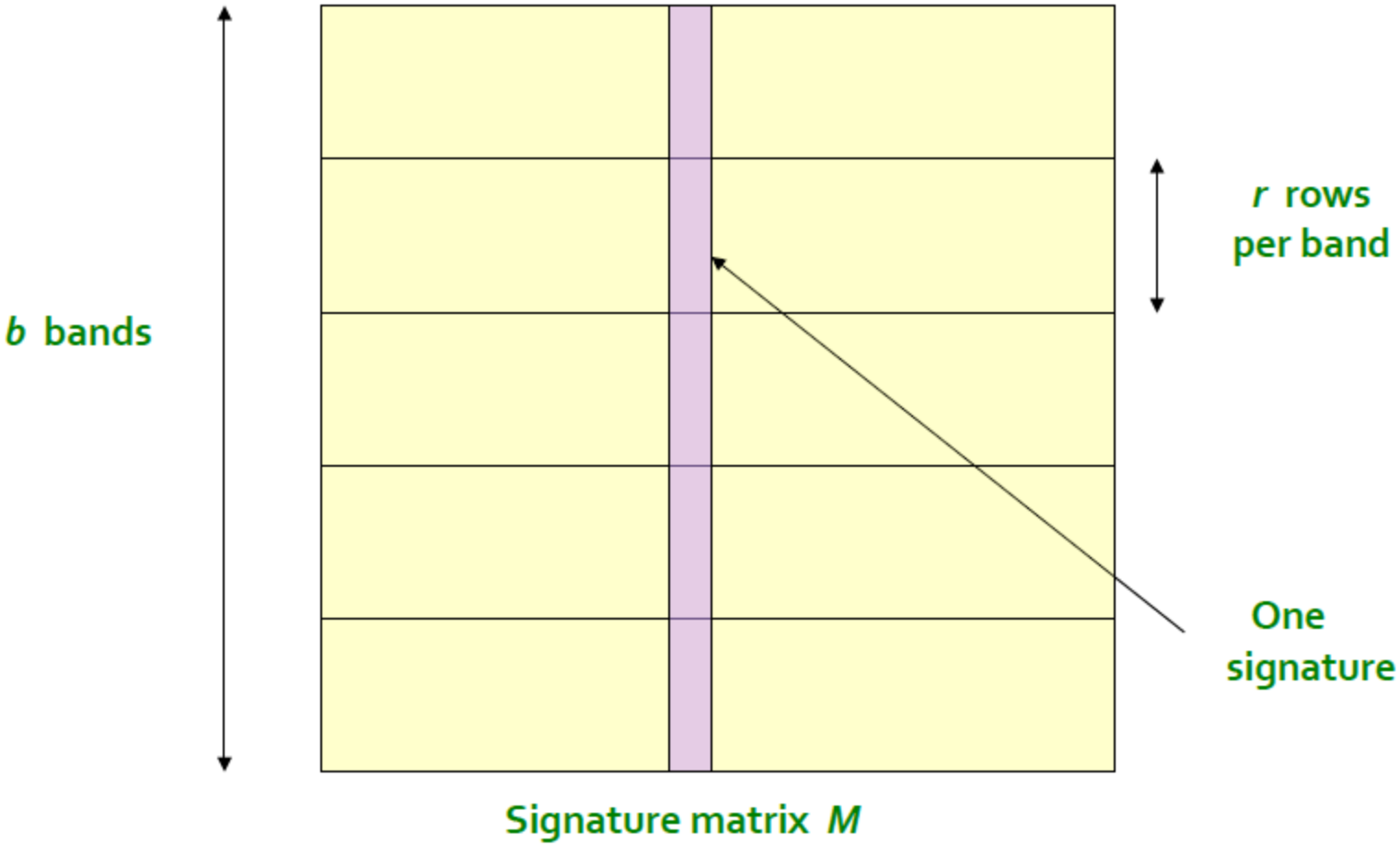
1-2	1-3	1-4	2-3
0	2/3	0	0

LSH for Min-hashing

- **Big idea: Hash columns of signature matrix *M* *several times***
- Arrange that (only) similar columns are likely to hash to the same bucket, with high probability
- **Candidate pairs are those that hash to the same bucket**

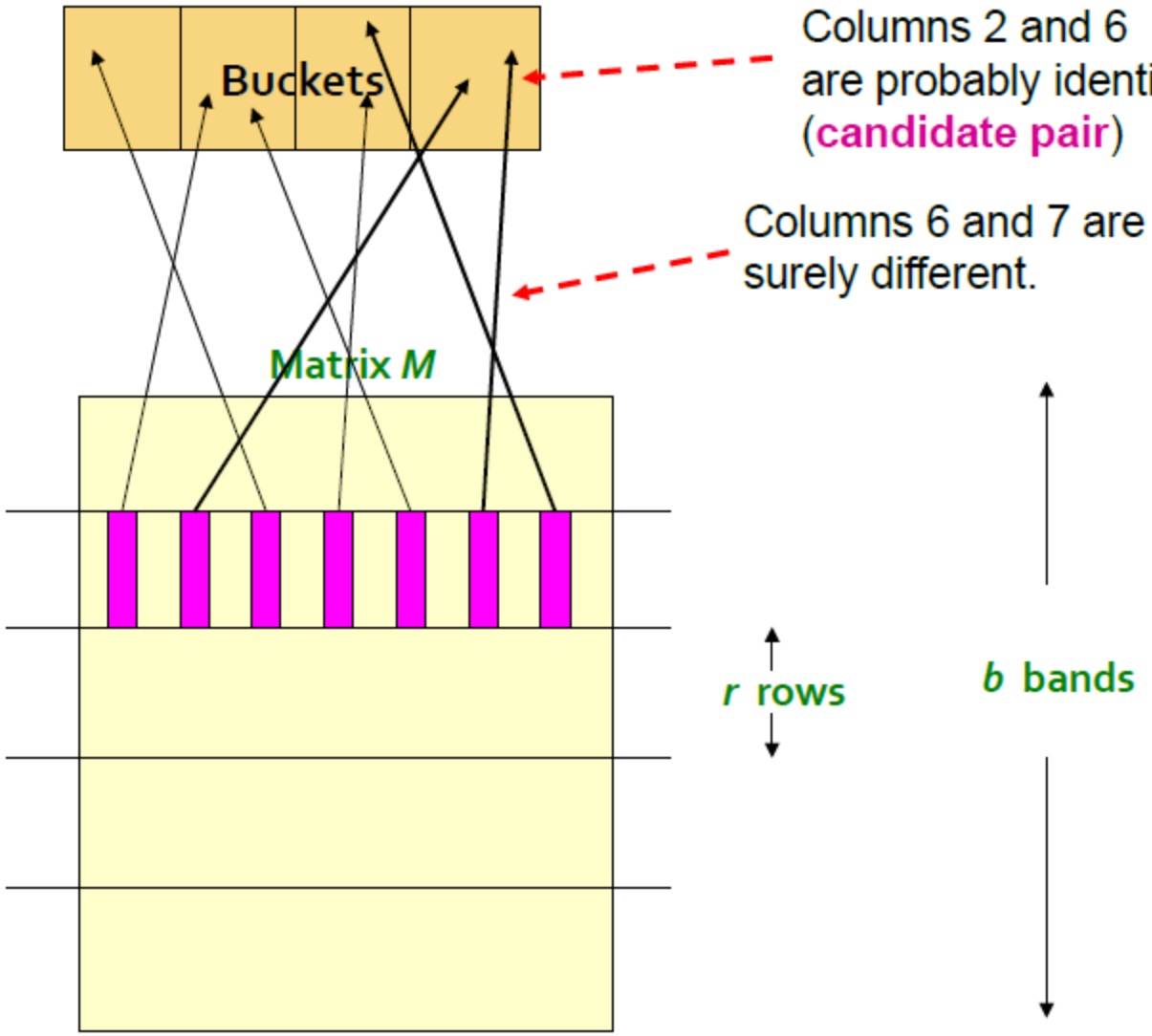
Partition M into b bands

2	1	4	1
1	2	1	2
2	1	2	1



- Divide matrix ***M*** into ***b bands*** of ***r rows***
- For each band, hash its portion of each column to a hash table with ***k buckets***
  - *Make k as large as possible*
- ***Candidate column pairs are those that hash to the same bucket for ≥ 1 band***
- Tune ***b and r*** *to catch most similar pairs, but few non-similar pairs*

Hashing bands



- Divide matrix ***M*** into ***b bands*** of ***r rows***
- For each band, hash its portion of each column to a hash table with ***k buckets***
  - *Make k as large as possible*
- ***Candidate column pairs are those that hash to the same bucket for ≥ 1 band***
- Tune ***b and r*** *to catch most similar pairs, but few non-similar pairs*

Simplifying Assumption

- There are **enough buckets** that columns are unlikely to hash to the same bucket unless they are identical in a particular band
- Hereafter, we assume that "**same bucket**" means **"identical in that band"**
- Assumption needed only to simplify analysis, not for correctness of algorithm

Example of Bands

Assume the following case:

- Suppose 100,000 columns of ***M*** (100k docs)
- Signatures of 100 integers (rows)
- Therefore, signatures take 40Mb
- Choose ***b*** = 20 bands of ***r*** = 5 integers/band

2	1	4	1
1	2	1	2
2	1	2	1

- **Goal:** Find pairs of documents that are at least ***s*** = *0.8* similar

Example

- Given 3 documents with their signatures:
  - S1 = [1,4,3,2,7,6]
  - S2 = [1,4,3,8,5,6]
  - S3 = [2,9,5,3,4,0]
- Choose b = 3 and r = 2.
- Hash function: h(band) =. (Sum of entries) mod 10
- Apply LSH to find all similar bands.

C_1, C_2 are 80% Similar

2	1	4	1
1	2	1	2
2	1	2	1

- **Find pairs of ≥ *s*=0.8 similarity, set b=20, r=5**
- **Assume:** sim(C_1, C_2) = 0.8
  - Since sim(C_1, C_2) ≥ **s**, we want C_1, C_2 to be a **candidate pair**: We want them to hash to at **least 1 common bucket** (at least one band is identical)
- **Probability C_1, C_2 identical in one particular band:** $(0.8)^5 = 0.328$
- Probability C_1, C_2 are ***not*** similar in all of the 20 bands: $(1\text{-}0.328)^{20} = 0.00035 \approx 1 / 2857 \approx$ **1 / 3000**
  - i.e., about 1/3000th of the 80%-similar column pairs are **false negatives** (we miss them)
  - **We would find 99.965% pairs of truly similar documents**

C_1, C_2 are 30% Similar

2	1	4	1
1	2	1	2
2	1	2	1

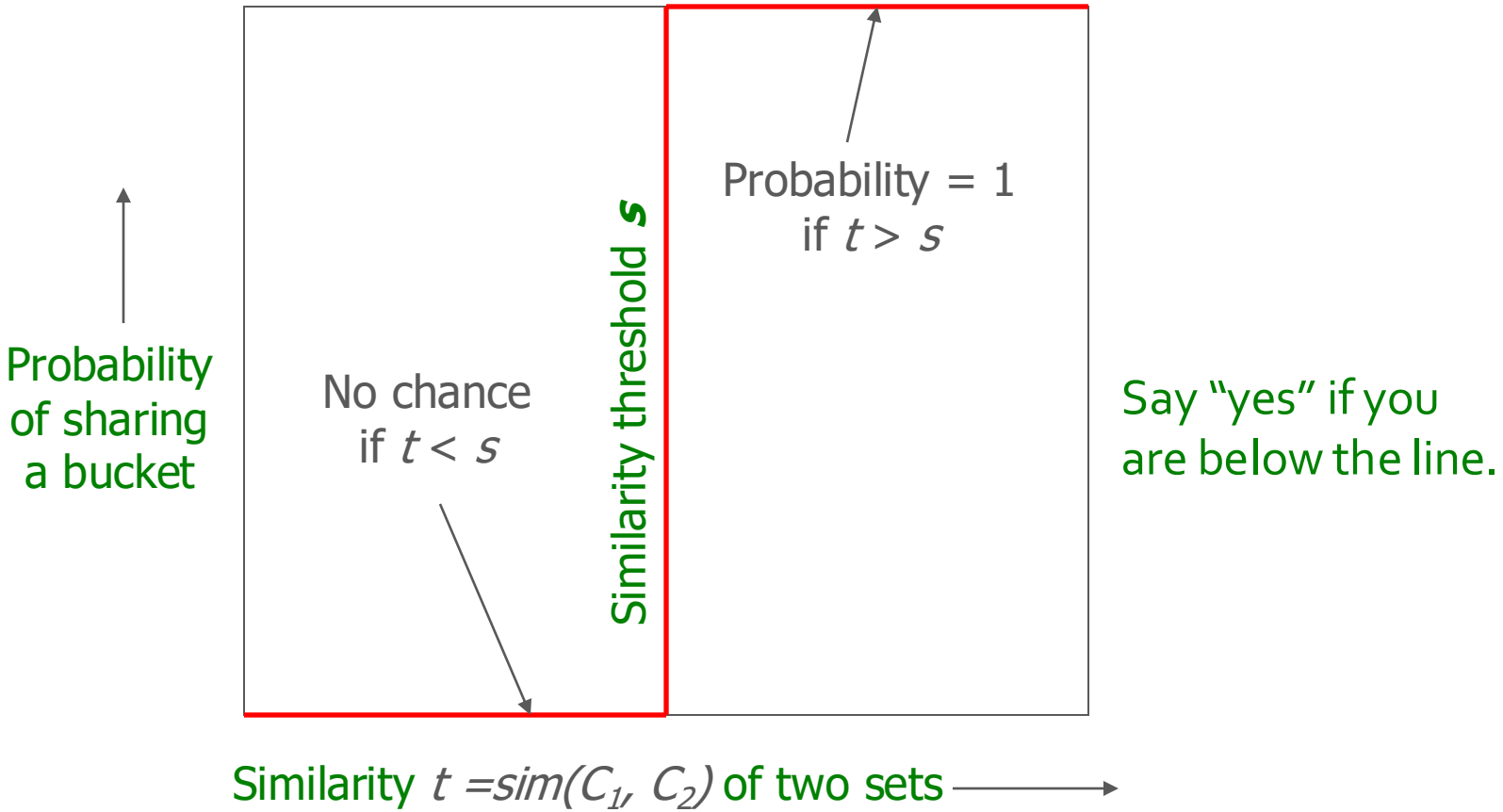
- **Find pairs of ≥ *s*=0.8** similarity, set **b**=20, **r**=5
- **Assume:** sim(C_1, C_2) = 0.3
  - Since sim(C_1, C_2) < **s** we want C_1, C_2 to hash to **NO common buckets** (all bands should be different)
- **Probability C_1, C_2 identical in one particular band:** $(0.3)^5 = 0.00243$
- Probability C_1, C_2 identical in at least 1 of 20 bands: $1 - (1 - 0.00243)^{20} = 0.0474$
  - In other words, approximately 4.74% pairs of docs with similarity 0.3% end up becoming **candidate pairs**
    - They are **false positives** since we will have to examine them (they are candidate pairs) but then it will turn out their similarity is below threshold **s**

LSH Involves a Tradeoff

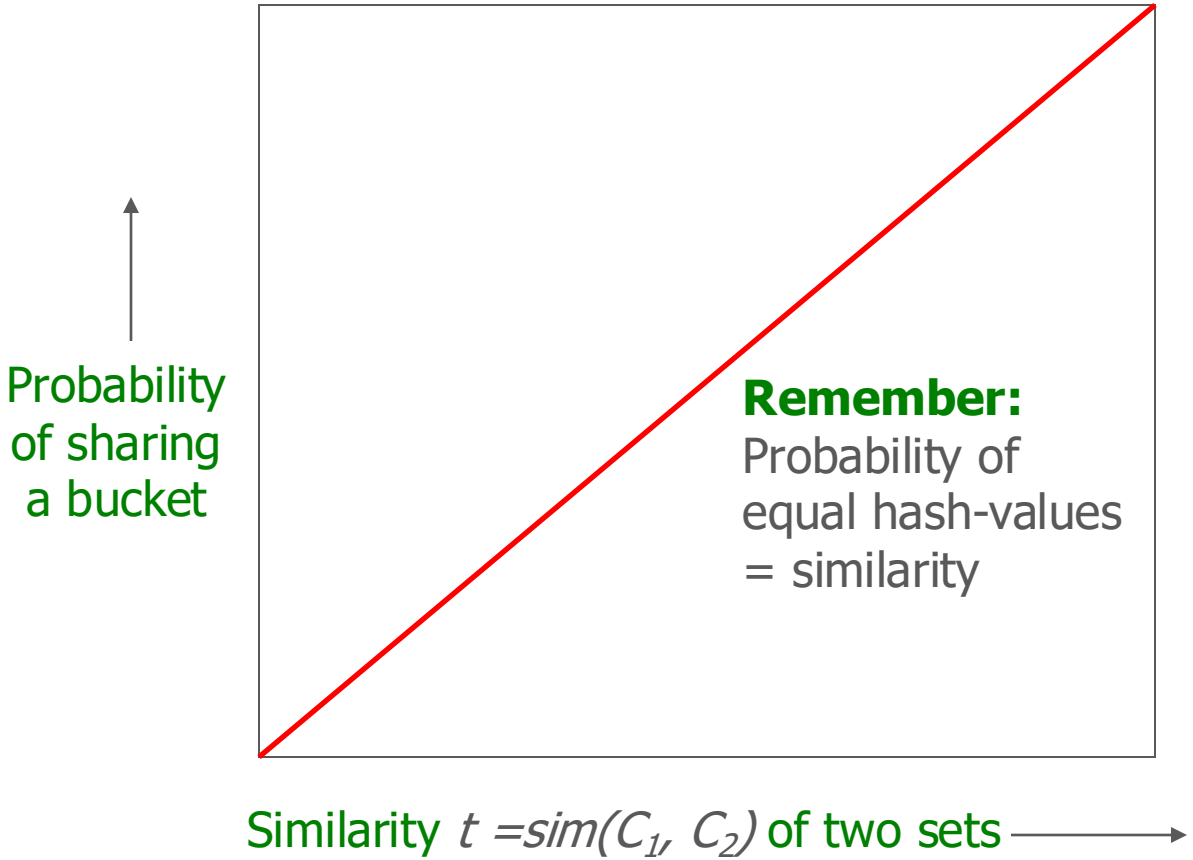
2	1	4	1
1	2	1	2
2	1	2	1

- **Pick:**
  - The number of Min-Hashes (rows of ***M***)
  - The number of bands ***b***, and
  - The number of rows ***r*** per band to balance false positives/negatives
    - Note, M=b*r
- **Example:** If we had only 15 bands of 5 rows, the number of false positives would go down, but the number of false negatives would go up

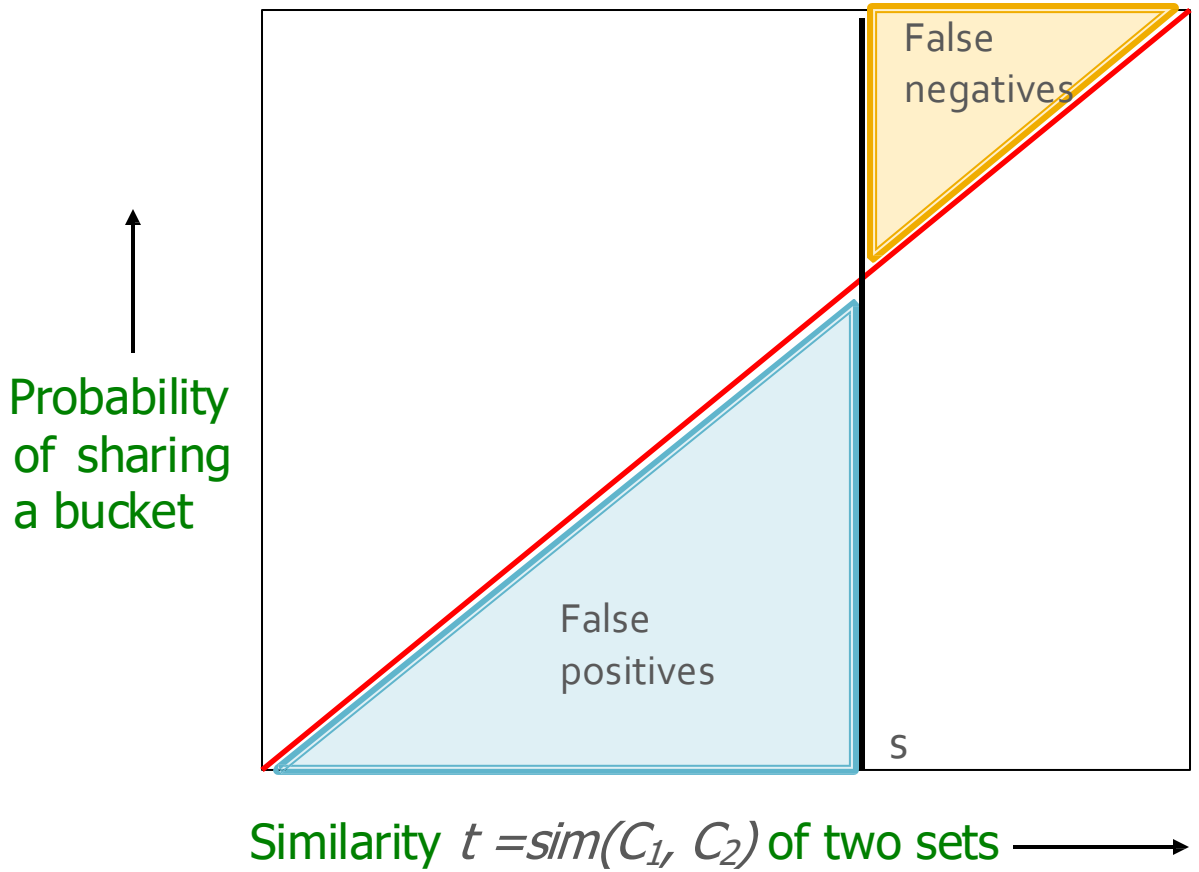
Analysis of LSH – What We Want



What 1 Band of 1 Row Gives You



What 1 Band of 1 Row Gives You (2)

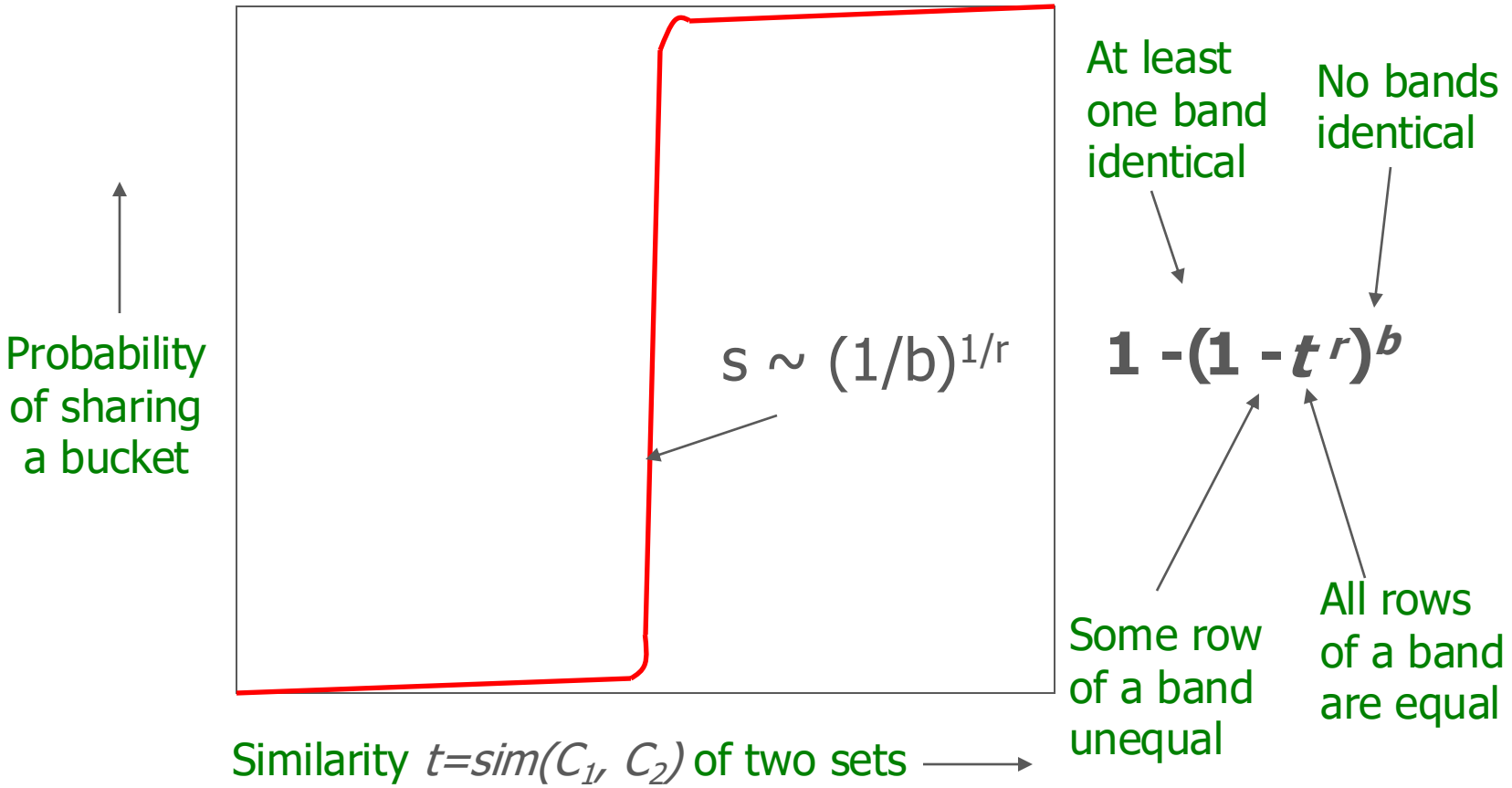


Say "yes" if you are below the line.

b bands, *r* rows/band

- Columns C_1 and C_2 have similarity ***t***
- Pick any band (***r*** rows)
  - Prob. that all rows in band equal = t^r
  - Prob. that some row in band unequal = $1 - t^r$
- Prob. that no band identical = $(1 - t^r)^b$
- Prob. that at least 1 band identical = $1 - (1 - t^r)^b$

What *b* Bands of *r* Rows Gives You



Example: $b = 20$; $r = 5$

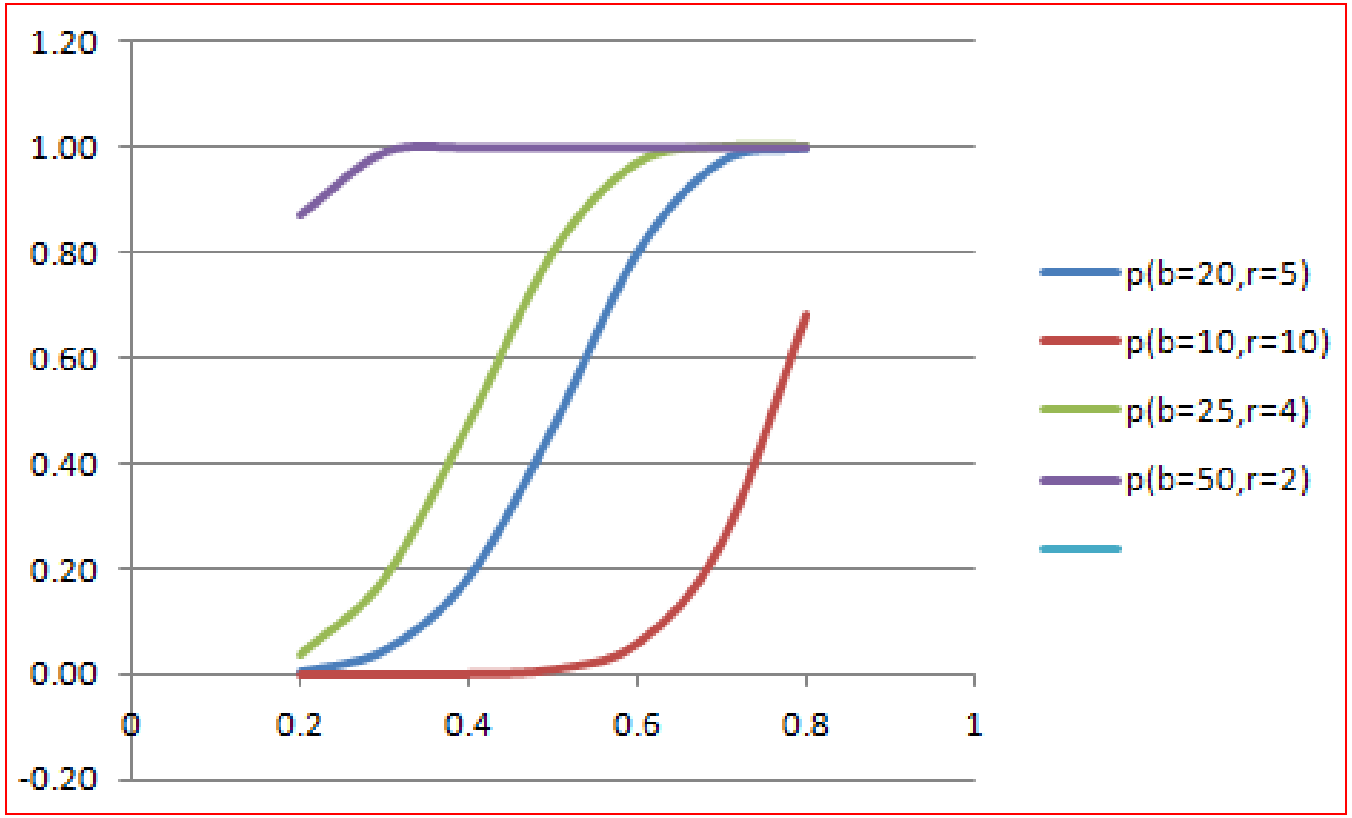
- **Similarity threshold s**
- **Prob. that at least 1 band is identical:**

s	$1-(1-s^r)^b$
.2	.006
.3	.047
.4	.186
.5	.470
.6	.802
.7	.975
.8	.9996

Example of S-curve

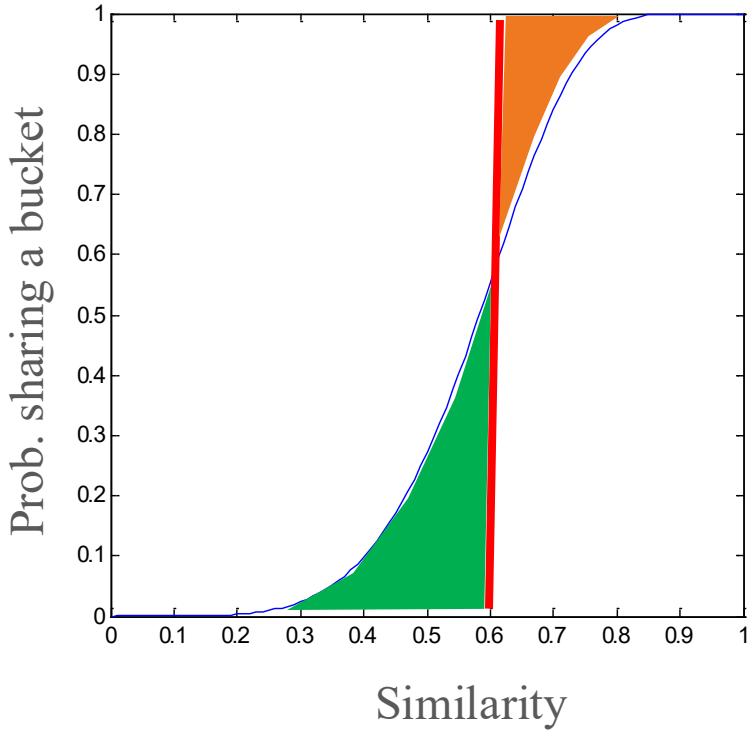
- **Similarity threshold s**
- **Prob. that at least 1 band is identical:**

s	p(b=20, r=5)	p(b=10, r=10)	p(b=25, r=4)	p(b=50, r=2)
0.2	0.0064	0.0000	0.0392	0.8701
0.3	0.0475	0.0001	0.1840	0.9910
0.4	0.1860	0.0010	0.4771	0.9998
0.5	0.4701	0.0097	0.8008	1.0000
0.6	0.8019	0.0588	0.9689	1.0000
0.7	0.9748	0.2491	0.9990	1.0000
0.8	0.9996	0.6789	1.0000	1.0000



Picking *r* and *b*: The S-curve

- **Picking *r* and *b* to get the best S-curve**
  - 50 hash-functions (r=5, b=10)



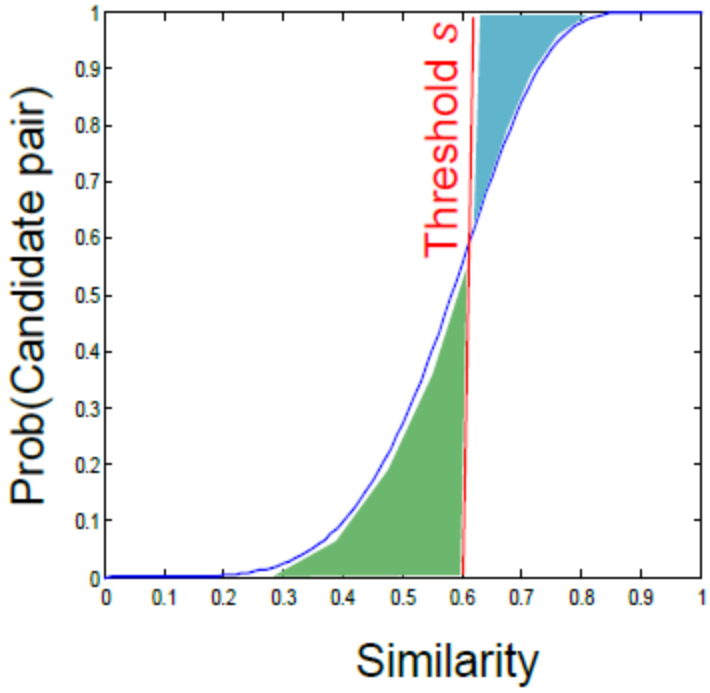
Orange area: False Negative rate
Green area: False Positive rate

Example

- **Assume the following case:**
  - Suppose 100,000 columns of ***M (100k docs)***
  - Signatures of 100 integers (rows)
  - Therefore, signatures take 40Mb
- Choose ***b = 20 bands of r = 5 integers/band***
- **Goal: Find pairs of documents that are at least *s = 0.8 similar***

Picking r and b: The S-curve

- **Picking *r and b to get desired performance 50 hash-functions (r = 5, b = 10)***
- **Blue area *X*: False Negative rate**
- These are pairs with sim > s but the X fraction won't share a band and then will never become candidates. This means we will never consider these pairs for (slow/exact) similarity calculation!
- **Green area Y: False Positive rate**
- These are pairs with sim < s but we will consider them as candidates. This is not too bad, we will consider them for (slow/exact) similarity computation and discard them.



LSH applications

- Audio processing
- Image/Video processing
- Security, privacy
- Blockchain
- Data mining
- Text/document processing
- Biological sciences
- Geological sciences
- Graph processing
- Machine learning
- Healthcare
- Plagiarism/Near-Duplicate Detection
- Networks
- Software testing
- Ontology Matching
- Social Media and Community Detection
- Time Series
- Robotics

[A Survey on Locality Sensitive Hashing Algorithms and their Applications, 2021, https://arxiv.org/pdf/2102.08942.pdf]

LSH Summary

- Tune ***M, b, r*** to get almost all pairs with similar signatures, but eliminate most pairs that do not have similar signatures

- Check in main memory that **candidate pairs** really do have **similar signatures**

- **Optional:** In another pass through data, check that the remaining candidate pairs really represent similar documents

Summary: 3 Steps

- **Shingling:** Convert documents to sets
  - We used hashing to assign each shingle an ID
- **Min-Hashing:** Convert large sets to short signatures, while preserving similarity
  - We used **similarity preserving hashing** to generate signatures with property $\mathbf{Pr}[h_\pi(C_1) = h_\pi(C_2)] = sim(C_1, C_2)$
  - We used hashing to get around generating random permutations
- **Locality-Sensitive Hashing:** Focus on pairs of signatures likely to be from similar documents
  - We used hashing to find **candidate pairs** of similarity $\geq$ **s**

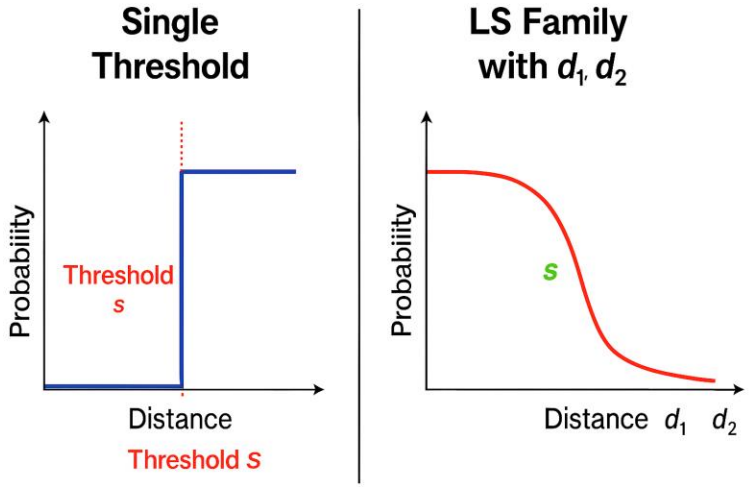
Locality sensitive (LS) families

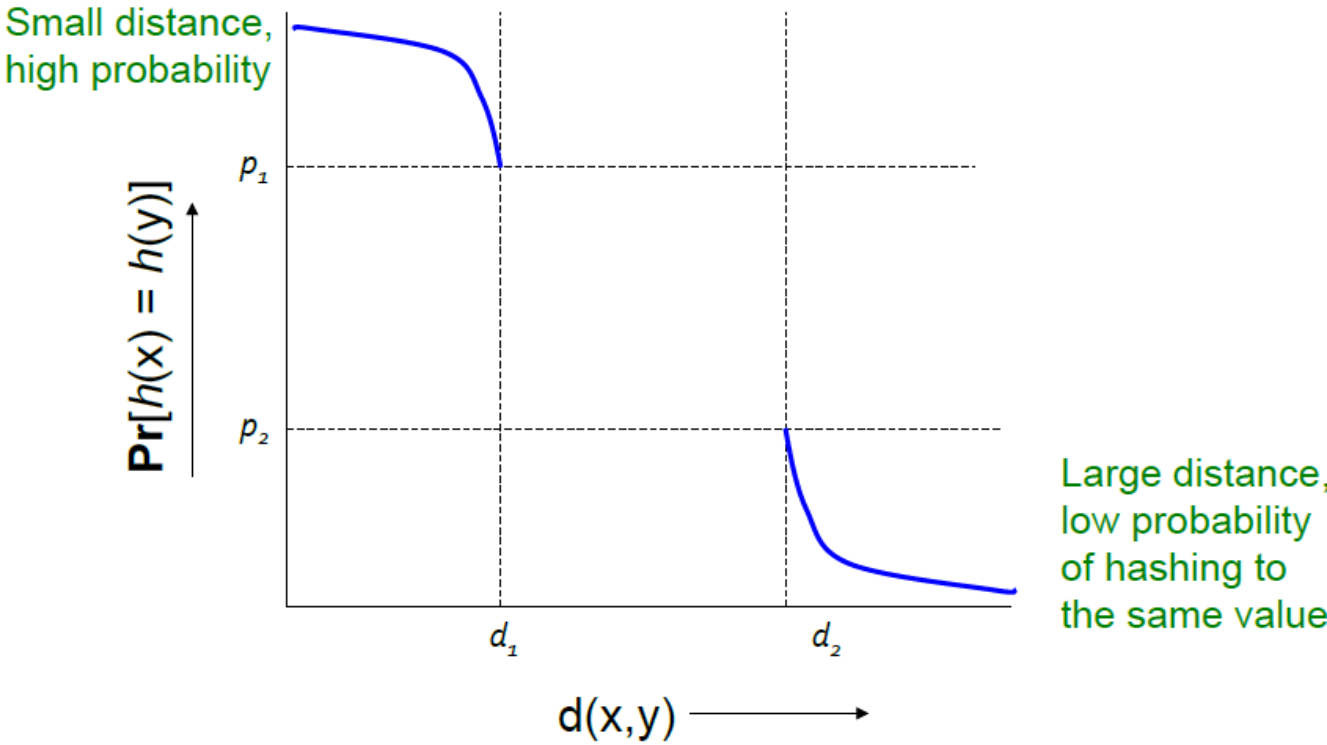
Families of Hash Functions

- **For Min-Hashing signatures, we got a Min-Hash function for each permutation of rows**
- A "hash function" is any function that takes *two elements and says whether they are "equal"*
  - h(x) = h(y) means "h says x and y are equal"
- A family of hash functions is any set of hash functions from which we can pick one at random efficiently
  - Example: The set of Min-Hash functions generated from permutations of rows

Locality sensitive Families

- Suppose we have a space ***S*** of points with a distance measure ***d(x,y)***
- A family H of hash functions is said to be **(d_1, d_2, p_1, p_2)-sensitive** if for any ***x*** and ***y*** in ***S***:
  - **If d(x, y) < d_1, P[h(x) = h(y)] ≥ p_1**
  - **If d(x, y) > d_2, P[h(x) = h(y)] ≤ p_2**





Min-Hash example

- S = space of all sets,
- d = Jaccard distance,
- H is family of Min-Hash functions for all permutations of rows
- Then for any hash function h∈ H:

$$\mathbf{Pr[h(x) = h(y)] = 1 - d(x, y)}$$

- Simply restates theorem about Min-Hashing in terms of distances rather than similarities

Min-Hash example

- Claim: Min-hash *H is a (1/3, 2/3, 2/3, 1/3)-sensitive family for S and d.*

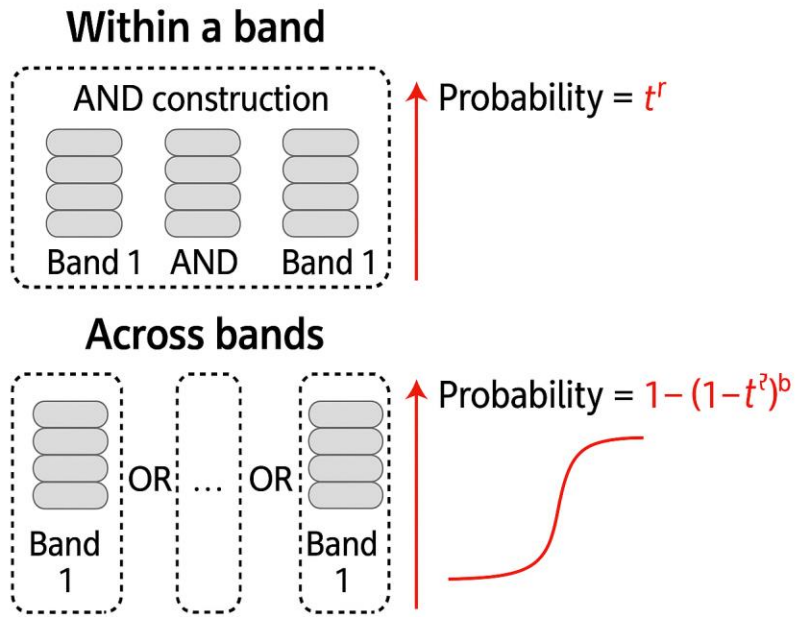
If distance < 1/3 (so similarity ≥ 2/3)

Then propability that Min-Hash values agree is ≥ 2/3

s_1 **s_2**

- For Jaccard similarity, Min-Hashing gives a $(d_1,d_2,(1-d_1),(1-d_2))$-*sensitive family for any* $d_1<d_2$
- Theory leaves unknown **what happens** to pairs that are at distance between d_1 *and* d_2

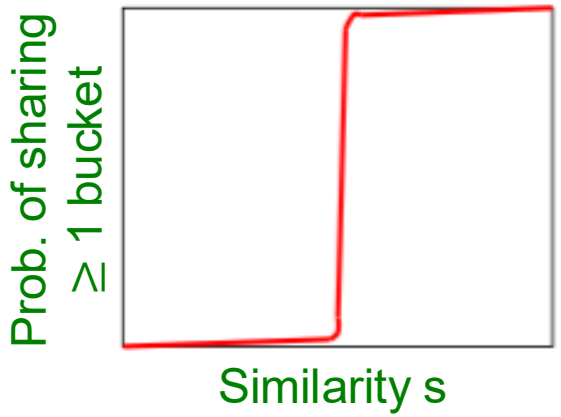
Amplifying a LS-familty



- Can we reproduce the "S-curve" effect we saw before for any LS family?

- The "**bands**" technique we learned for signature matrices carries over to this more general setting
- **Can do LSH with any (d_1, d_2, p_1, p_2)-*sensitive* family!**

- **Two constructions:**
  - **AND** construction like "rows in a band"
  - **OR** construction like "many bands"



AND of Hash Functions

- Given family $\boldsymbol{H}$, construct family $\boldsymbol{H'}$ consisting of r functions from $\boldsymbol{H}$
- For $\boldsymbol{h = [h_1,\ldots,h_r]}$ in **H'**, we say

  $\mathbf{h(x) = h(y)}$ if and only if $\mathbf{h_i(x) = h_i(y)}$ for **all** $\boldsymbol{i}$ $(\boldsymbol{1 \le i \le r})$
  - Note this corresponds to creating a band of size r
- **Theorem:** If H is $\boldsymbol{(d_1, d_2, p_1, p_2)}$-sensitive, then

  H' is $\boldsymbol{(d_1,d_2, (p_1)^r, (p_2)^r)}$-sensitive

Also lowers probability for small distances (Bad)

Lowers probability for large distances (Good)

- **Proof:** Use the fact that $\boldsymbol{h_i}$'s are **independent**

OR of Hash Functions

- Given family H, construct family H' consisting of b functions from H

- For $\boldsymbol{h = [h_1,\ldots,h_b]}$ in $\boldsymbol{H'}$,

  $\mathbf{h(x) = h(y)}$ if and only if $\mathbf{h_i(x) = h_i(y)}$ for **at least 1**

- **Theorem:** If H is (d_1, d_2, p_1, p_2)-sensitive,

  then H' is $\mathbf{(}d_1, d_2, \mathbf{1-(1-}p_1\mathbf{)^b, 1-(1-}p_2\mathbf{)^b)}$-sensitive

Raises probability for small distances (Good)

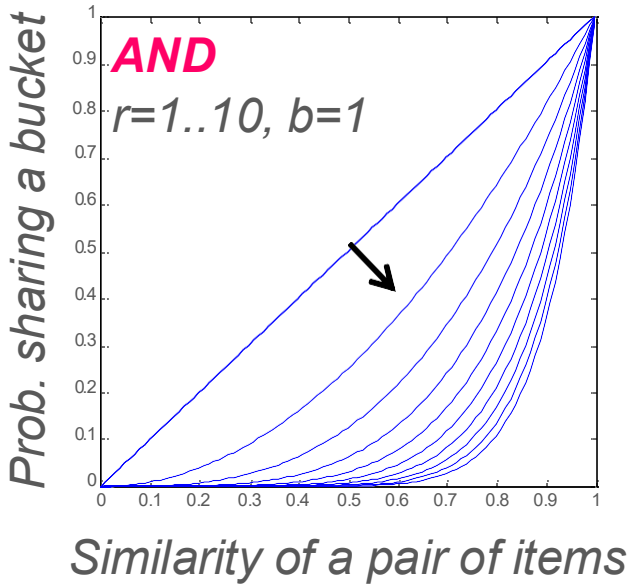
Raises probability for large distances (Bad)

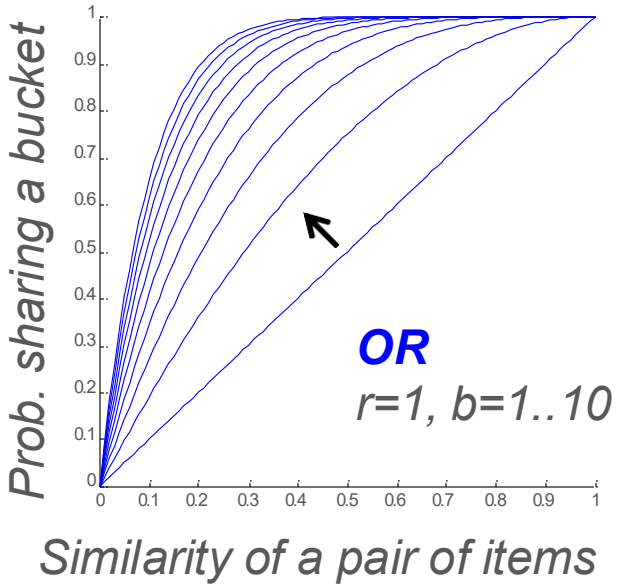
- **Proof:** Use the fact that h_i's are **independent**

Effect of AND and OR Constructions

- **AND** makes all probs. shrink, but by choosing ***r*** correctly, we can make the lower prob. approach 0 while the higher does not

- **OR** makes all probs. grow, but by choosing ***b*** correctly, we can make the higher prob. approach 1 while the lower does not





Combine of AND and OR Constructions

- By choosing ***b*** and ***r*** correctly, we can make the lower probability approach 0 while the higher approaches 1

- As for the signature matrix, we can use the AND construction followed by the OR construction
  - Or vice-versa
  - Or any sequence of AND's and OR's alternating

Composing Constructions

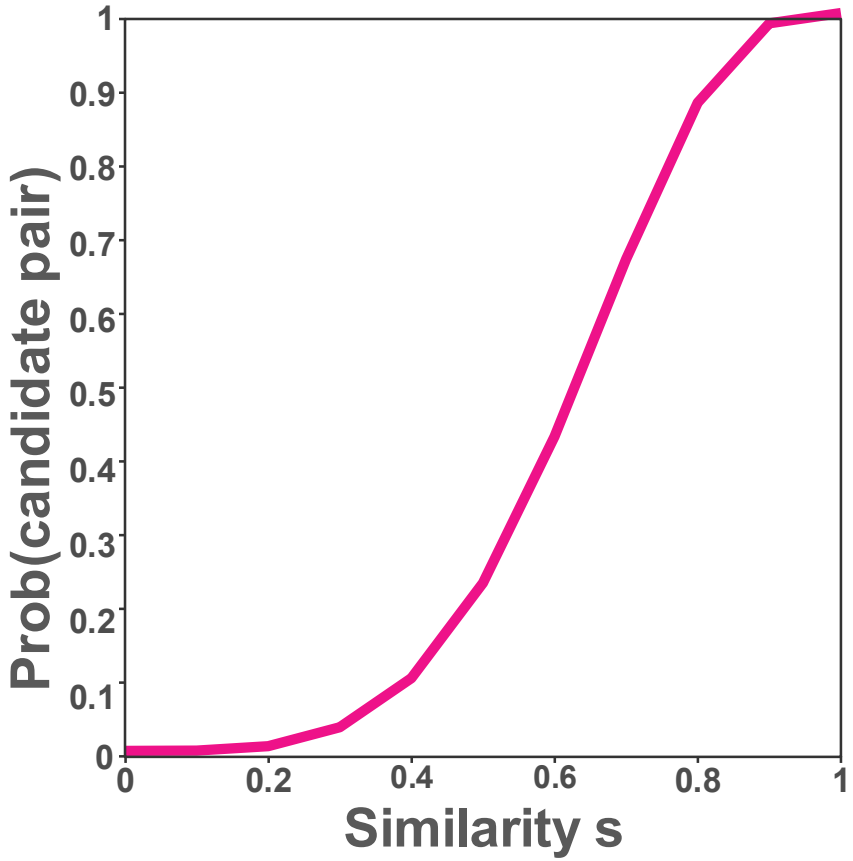
- *r*-way AND followed by *b*-way OR construction
  - Exactly what we did with Min-Hashing
    - **AND:** If bands match in **all *r*** values hash to same bucket
    - **OR:** Cols that have ≥ 1 common bucket -> **Candidate**
- Take points ***x*** and ***y*** s.t. ***Pr[h(x) = h(y)] = s***
  - ***H*** will make **(x,y)** a candidate pair with prob. **S**
- Construction makes **(x,y)** a candidate pair with probability $1-(1-s^r)^b$.

The S-Curve!

- **Example:** Take **H** and construct **H'** by the **AND** construction with ***r* = 4**.
  Then, from **H'**, construct **H"** by the **OR** construction with ***b* = 4**

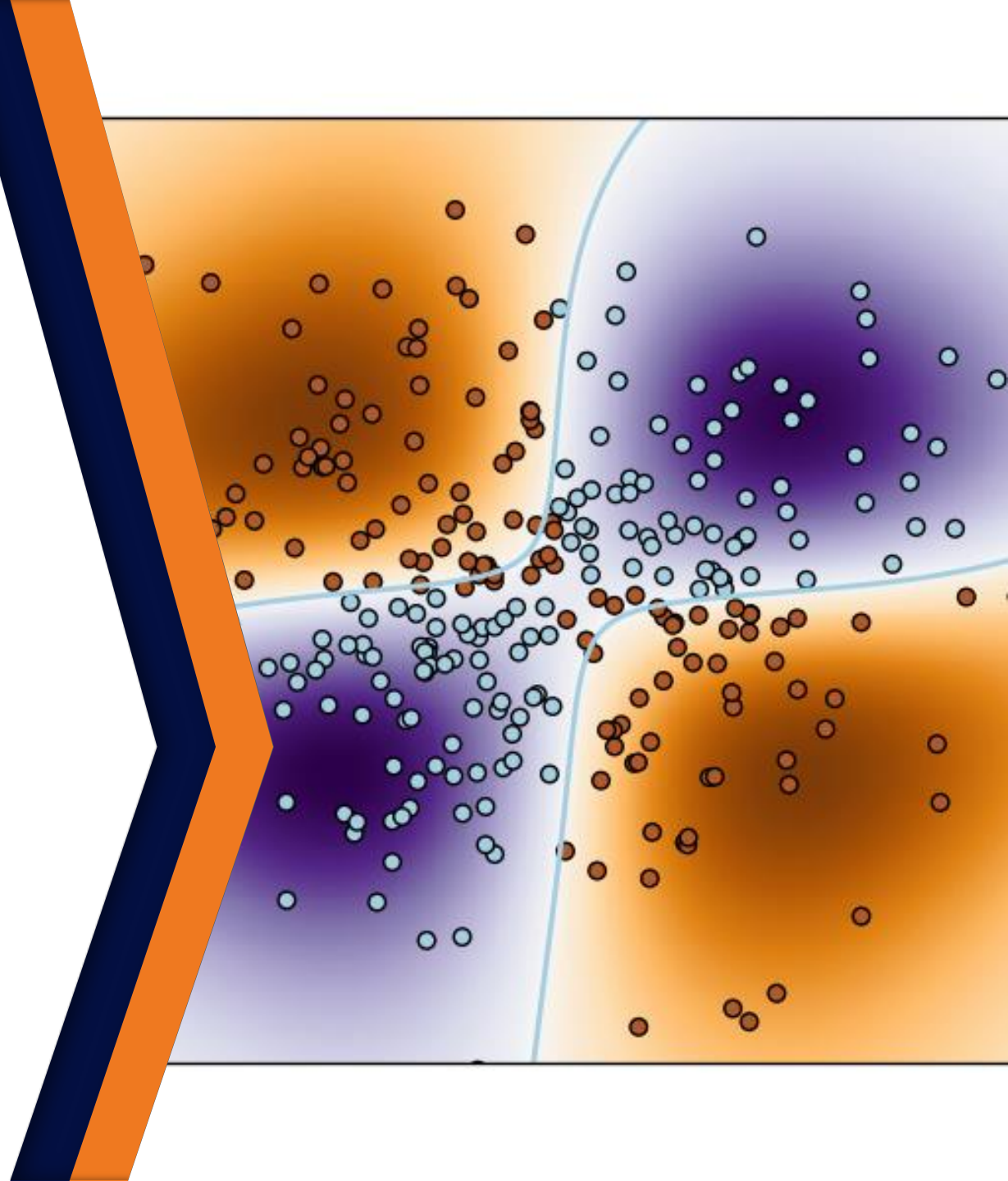
Table for Function $1-(1-s^4)^4$

s	$p=1-(1-s^4)^4$
.2	**.0064**
.3	.0320
.4	.0985
.5	.2275
.6	.4260
.7	.6666
.8	**.8785**
.9	.9860



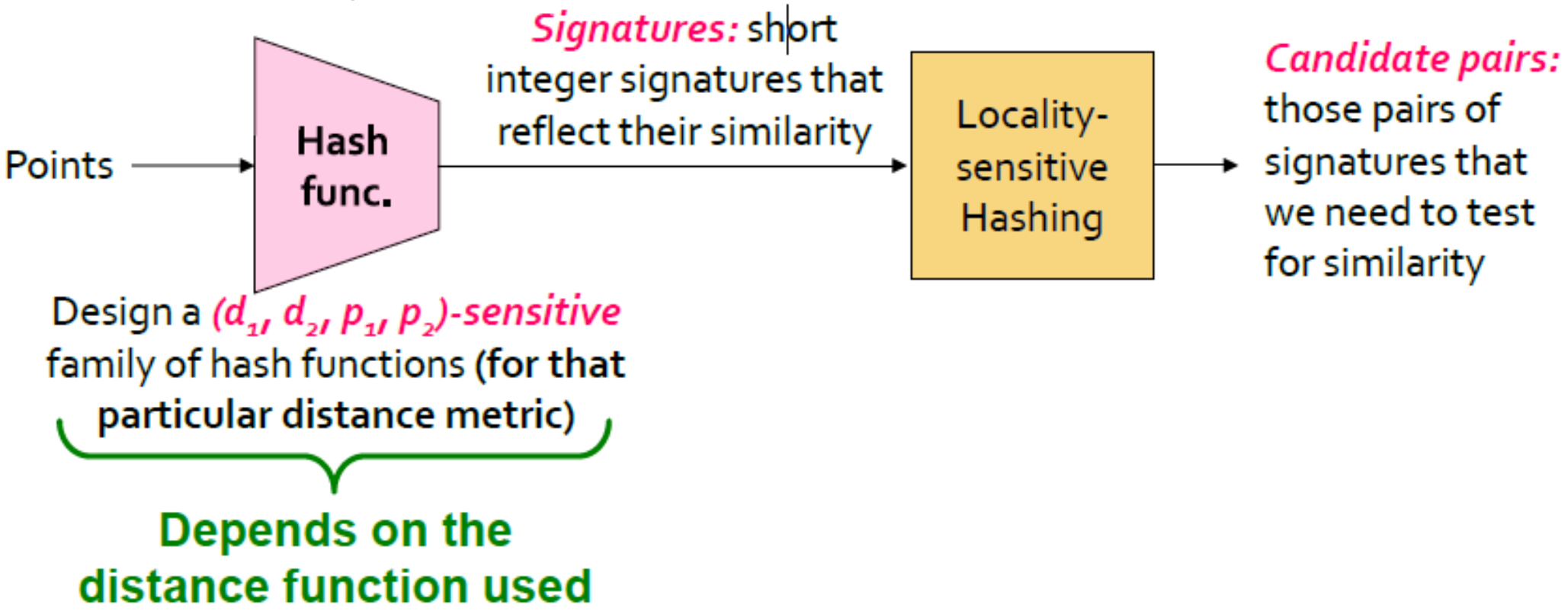
r = 4, b = 4 transforms a (.2,.8,.8,.2)-sensitive family into a (.2,.8,.8785,.0064)-sensitive family.

LSH for other distance



LSH for other distance

- **Cosine distance: Random hyperplanes**
- **Euclidean distance: Project on lines**

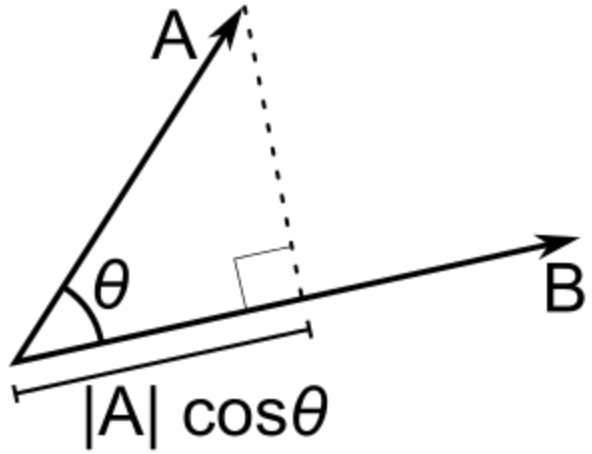


Cosine distance

- ***Cosine distance = angle between vectors from the origin to the points in question***

  d(A, B) = θ = arccos(A·B / ‖A‖·‖B‖)

- Has range **0…π** (equivalently 0...180°)
- Can divide θ by **π** to have distance in range 0…1
- **Cosine similarity = 1-d(A,B)**



$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|} = \frac{\sum_{i=1}^{n} A_i \times B_i}{\sqrt{\sum_{i=1}^{n} (A_i)^2} \times \sqrt{\sum_{i=1}^{n} (B_i)^2}}$$

LSH for Cosine distance

- For **cosine distance, there is a technique called Random Hyperplanes**
  - Technique similar to Min-Hashing
- Random Hyperplanes method is a *$(d_1, d_2, (1-d_1/\pi), (1-d_2/\pi))$-sensitive family for any d_1 and d_2*
  - **Reminder: *(d_1, d_2, p_1, p_2)-sensitive***
    - *If $d(x,y) < d_1$, then prob. that $h(x) = h(y)$ is at least p_1*
    - If *$d(x,y) > d_2$, then prob. that $h(x) = h(y)$ is at most p_2*
- For points **x and y**,

  $\Pr[h(x) = h(y)] = 1 - d(x,y) / \pi$

LSH for Cosine distance (example)

$$h_r = \begin{cases} 1, \text{if r.v} \geq 0 \\ 0, \text{if r.v} < 0 \end{cases}$$

r: a random vector have the same dimension with v

Given 3 vectors: v1 = [1,1], v2 = [1,0.9], v3 = [-1,-1] r = [0.5,-0.2]

$h_r(v1) = 1, rv1 = 0.3 > 0$
$h_r(v2) = 1, rv2 = 0.32 > 0$
$h_r(v1) = 0, rv3 = -0.3 < 0$

=> v1, v2 go to the same bucket

LSH for Euclidean distance

- **Simple idea: Hash functions correspond to lines**
- Partition the line into buckets of size ***a***
- **Hash each point to the bucket containing its projection onto the line**
- **Nearby points are always close; distant points are rarely in same bucket**

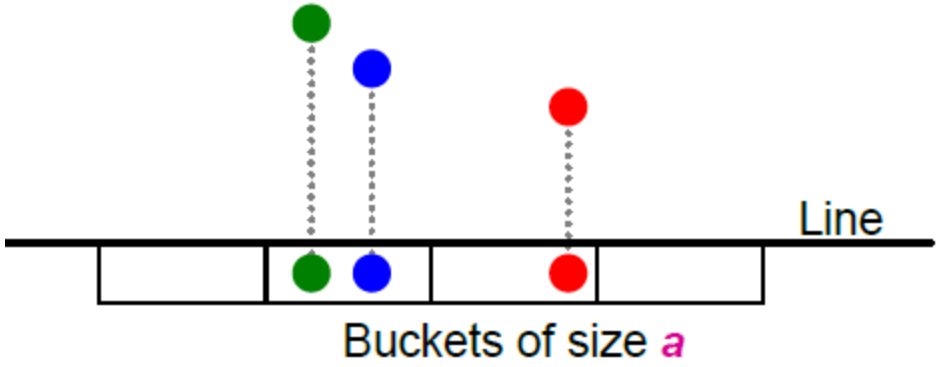
$$\mathrm{d}(\mathbf{p}, \mathbf{q}) = \mathrm{d}(\mathbf{q}, \mathbf{p}) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \cdots + (q_n - p_n)^2} = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}.$$

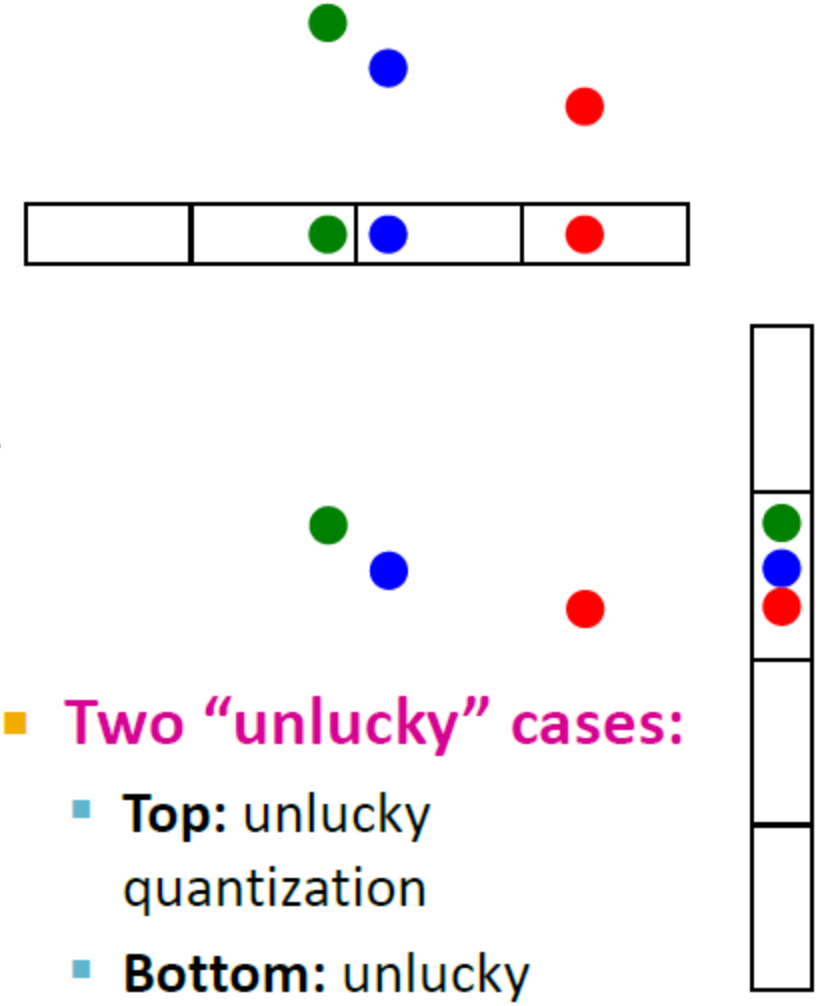
Hash function used for euclidean distance

$$h_{a,b}(v) = \left\lfloor \frac{a \cdot v + b}{w} \right\rfloor$$

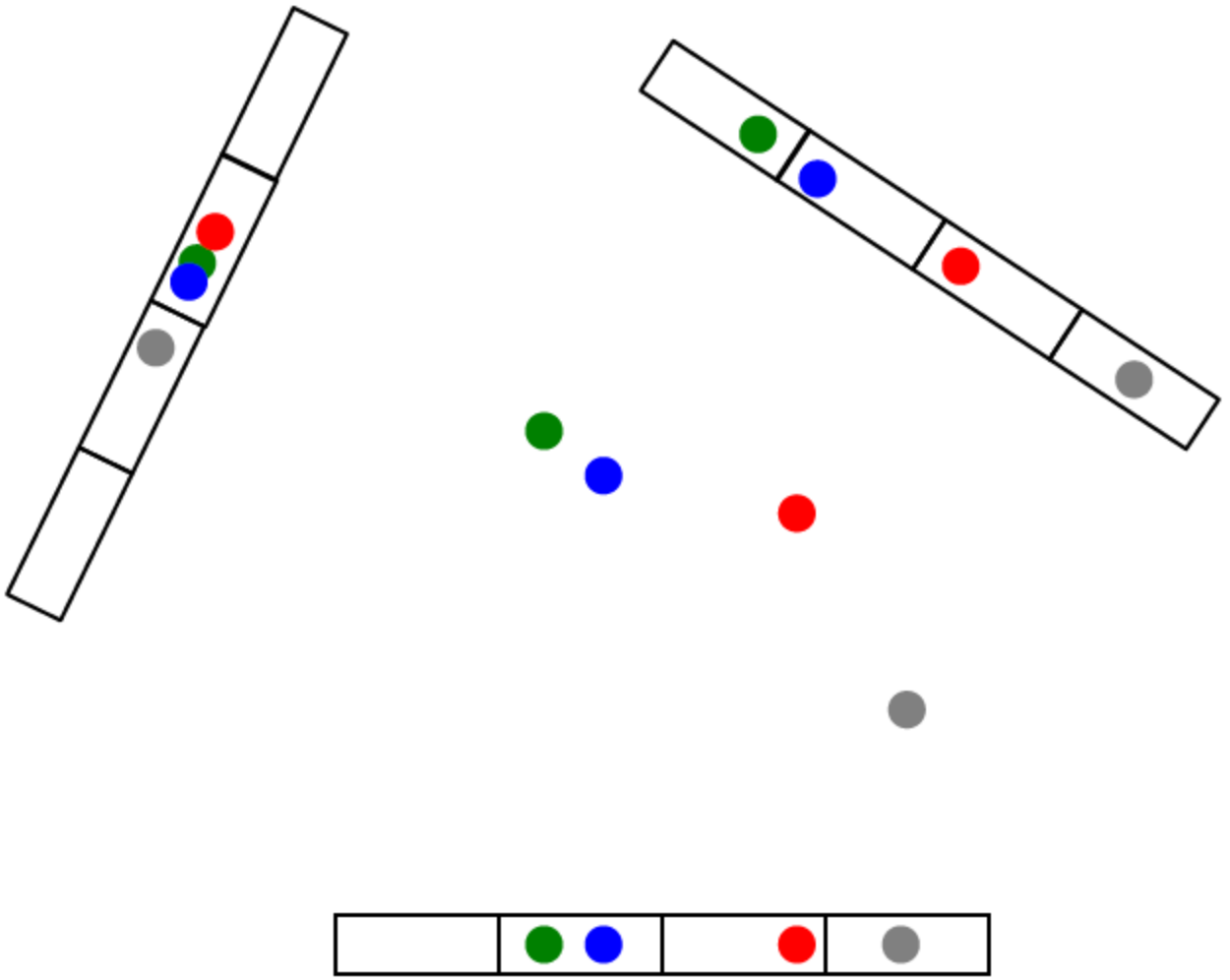
v: the input feature vector
a: a random vector, $\sim \mathcal{N}(0,1)$
b$\in$[0,w): a random offset uniformly sampled
w: bucket width, controlling the sensitivity of the hashing

LSH for Euclidean distance





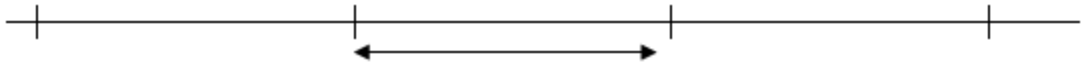
- **"Lucky" case:**
  - Points that are close hash in the same bucket
  - Distant points end up in different buckets
- **Two "unlucky" cases:**
  - **Top:** unlucky quantization
  - **Bottom:** unlucky projection



Projection of points

Points at distance ***d***

If ***d*** << ***a***, then the chance the points are in the same bucket is at least **1 – *d*/*a***.



Randomly chosen line

Bucket width ***a***

Methods for High Degrees of Similarity

Specialized techniques for finding pairs with very high Jaccard similarity

Overview of High-Similarity Methods

When similarity threshold is **very high** ($J \geq 90\%$):

- Standard minhashing and LSH become inefficient
- Too many minhash functions needed
- Too many LSH bands cause excessive false positives

Key insight: Similar sets have nearly equal sizes

Specialized techniques:

1. String Representation

Represent sets as strings for similarity techniques

2. Length Filtering

Compare only strings with similar lengths

3. Prefix Indexing

Index only beginning portions of strings

4. Position Information

Use character positions for selective indexes

5. Position + Length Indexing

Combine position and suffix length for precision

Exercise

Given 2 documents: D1=“abcab”, D2= "abceab"

Shingle	Hash value.
'ab'	101
'bc’	205
'cd’	309
'da'	401
'ce'	503
'ea'	607

With 3 hash functions: h1(x)=(2x+1) mod 10, h2(x)=(3x+2) mod 10, h3(x)=(5x+4) mod 10
Apply LSH to check if D1 is similar to D2, with b = 3, r = 1